



UNIVERSITÄT ULM

DOCTORAL THESIS

Trust Assessment and Propagation in Classifier Neural Network Pipelines based on Subjective Logic

Author

Koffi Ismael Ouattara

Supervisors

Frank Kargl
Theo Dimitrakos
Houda Labiod

Reviewers

Prof. Frank Kargl
Prof. Konrad Rieck

*A thesis submitted in fulfillment of the requirements
for the degree of Dr. rer nat.*

in the

Distributed Systems Group (VS)
Institut für Verteilte Systeme

August 24, 2026

Declaration of Authorship

I, Koffi Ismael Ouattara, declare that this thesis titled:

Trust Assessment and Propagation in Classifier Neural Network Pipelines based on Subjective Logic

is the product of my own independent work and that I have used no sources or materials other than those specified. The passages taken from other works, either verbatim or paraphrased in the spirit of the original quote, are identified in each individual case by indicating the source.

I further declare that all my academic work was written in line with the principles of proper academic research according to the official "Satzung der Universität Ulm zur Sicherung guter wissenschaftlicher Praxis" (University Statute for the Safeguarding of Proper Academic Practice).

Munich,

Koffi Ismael Ouattara, student number

Acknowledgements

This dissertation would not have been possible without the support of many people to whom I owe a great deal of gratitude.

First of all, I would like to thank my family. My father and mother have supported me since the very first days of my life, long before this PhD was even a distant thought, and their unwavering belief in me has been a quiet but constant source of strength throughout this journey. I also would like to express my deepest gratitude to my wife, Flan Ermine, whose patience, encouragement, and love have meant more than words can express. Having her by my side during the most demanding moments of this work made all the difference.

I am deeply grateful to my supervisors, Prof. Frank Kargl, Prof. Theo Dimitrakos, and Prof. Houada Labiod, for their guidance, availability, and trust throughout this research. Each of them brought a distinct and complementary perspective that shaped the direction and depth of this work in meaningful ways. I would also like to extend special thanks to Dr. Ioannis Krontiris, who, though not formally listed as a supervisor, was fully present in that role. His advice, feedback, and engagement were invaluable, and this dissertation is richer for his involvement. Beyond the scope of this PhD, Ioannis also took the time to share life advice that helped me grow as a person, and for that I am especially grateful.

Among my colleagues, I owe particular thanks to Hussein Joumaa, the best friend I had at the office. Our conversations about everything and nothing were a constant source of relaxation and perspective. I am also grateful to Dr. Ana Petrovska, who helped me improve my writing and collaborated with me on several internal projects and publications, and to Gabriele Gelardi, for the many insightful technical discussions on Subjective Logic and neural networks.

I also acknowledge the collaborators from outside Huawei, particularly from the University of Luxembourg: Prof. Gabriele Lenzini and Hüseyin Demirci, whose contributions enriched this work.

Finally, I am grateful to the friends who supported me both technically and morally throughout this PhD. Dr. Hugues Kouakou, Dro Sigui, and Adou Seka were fellow PhD students with whom it was invaluable to share the experience and realize we were all facing the same challenges. Alex N'guettia was always available and helped me with many tasks both personal and academic. Camara Yoane also helped with many personal matters and gave me the space to focus on this work. David Kragbe and Oumar Ouattara supported me since the very beginning of this journey.

UNIVERSITÄT ULM

Abstract

Institut für Verteilte Systeme

Dr. rer nat.

Trust Assessment and Propagation in Classifier Neural Network Pipelines based on Subjective Logic

by Koffi Ismael Ouattara

“There are no facts, only interpretations.”

Friedrich Nietzsche

Neural Networks (NNs) are increasingly deployed in high-stakes domains where the reliability of predictions has direct consequences. However most evaluation practices focus on aggregate performance metrics, which neither distinguish sources of uncertainty nor capture how trustworthiness propagates across the stages of a learning pipeline. This thesis addresses this gap by developing a formal, interpretable, and operational methodology for trust assessment in classifier neural network pipelines, grounded in Subjective Logic (SL). The thesis comprises four contributions. First, a General Trust Assessment Methodology formalizes trust evaluation as a three-phase process (expression generation, Atomic Trust Opinions quantification, and expression evaluation), and an optimized Trust Network Analysis algorithm ensures computational efficiency and consistency of SL inference over complex graph. Second, the Parallel Trust Assessment System (PaTAS) operationalizes trust assessment across the full Machine Learning (ML) pipeline: a trust assessment module quantifies reliability of the training dataset, and a trust propagation module mirrors neural network computations to carry trust from training dataset to parameters and from inputs through parameters to predictions, without modifying the underlying model. Third, a calibration-based trust formulation enables trust assessment in black-box settings using only observable model inputs and outputs. Fourth, an end-to-end evaluation on a 5G network energy classification task demonstrates that PaTAS trust opinions consistently discriminate between correct and incorrect predictions under controlled data quality variation, and produce signals coherent with the complementary calibration-trust view. Together, these contributions establish trust as a first-class, auditable object in neural network systems, and provide both the theoretical foundations and practical tools for its representation, quantification, and propagation.

Contents

Acknowledgements	v
Abstract	vii
List of Acronyms	xiii
List of Symbols and Notation	xvii
Publications	xxi
1 Introduction	1
1.1 Research Context and Motivation	1
1.2 Problem Statement and Research Questions	5
1.3 Contributions of the Thesis	8
1.4 Structure of the Thesis	11
2 Background and Related Work	13
2.1 Reasoning under uncertainty	13
2.2 Subjective Logic and Trust	19
2.3 Trust and Uncertainty in Neural Networks	35
2.4 Synthesis and Gap Summary	49
3 General Trust Assessment Methodology	53
3.1 Role of the General Trust Assessment Methodology	53
3.2 Core Concepts and Notation	54
3.3 Trust Assessment Methodology	57
4 Optimized Trust Network Analysis with Subjective Logic	61
4.1 Introduction	61
4.2 Parallel Non-Intersecting Path Subnetworks	64
4.3 Optimized DSPG Synthesis Criteria	66
4.4 Optimized DSPG Synthesis Algorithm	71
4.5 Optimized DSPG Analysis and PNPS-Based Reduction	76
4.6 Trust Discounting for Referral Paths	80
4.7 Evaluation	83
4.8 Summary & Conclusions	90
5 Parallel Trust Assessment System (PaTAS)	93
5.1 Introduction	93
5.2 PaTAS General Description	94
5.3 Functionalities	96
5.4 Summary & Conclusions	97

6	Assessment of AI Training Datasets Trustworthiness	99
6.1	Introduction	99
6.2	Sources of Uncertainty in Training Datasets Assessment	100
6.3	A Methodology for Trust Assessment of Training Datasets	102
6.4	Multi-stage Assessment: COMPAS	106
6.5	Sampling Stage, Dataset Level: Representativeness on GTSRB	110
6.6	Annotation Stage, Entry Level: Labeling accuracy on CIFAR-10H	116
6.7	Summary & Conclusions	119
7	Trust Propagation in Neural Network	123
7.1	Introduction	123
7.2	Foundations of Trust Propagation in Neural Networks	125
7.3	Parallel Trust Assessment System - Trust Propagation (PaTAS-TP): Ar- chitecture Overview	127
7.4	Trust Feedforward in a Simple Model: The Perceptron Case	130
7.5	Generalization to Deep NNs	132
7.6	Parameter-Trust Update	135
7.7	Theoretical Properties of PaTAS-TP	138
7.8	<u>Per-Sample Trust at Inference: Input Conformity and Composed Scores</u>	140
7.9	Evaluation and Results	141
7.10	Discussion of PaTAS-TP Findings	156
7.11	Summary & Conclusions	158
8	Calibration-Based Trust Assessment of NNs	161
8.1	Introduction	161
8.2	SL-Based Methodology for Calibration Trust Assessment	162
8.3	Experimental Evaluation	166
8.4	Summary & Conclusions	175
9	Use Case: PaTAS Applied to 5G Network Energy Management	177
9.1	Introduction and Motivation	177
9.2	Dataset and Experimental Setup	178
9.3	Trust Assessment: Assigning Opinions to Data	180
9.4	Trust Propagation: How Opinions Evolve During Training in PaTAS	183
9.5	Noise Robustness: PaTAS as a Confidence Signal Under Data Quality Variation	185
9.6	Calibration Quantification using Subjective Logic	191
9.7	Discussion	197
9.8	Summary & Conclusions	201
10	Conclusion and Future Work	203
10.1	Synthesis of Contributions	203
10.2	What This Thesis Does and Does Not Address	205
10.3	Limitations and Directions for Future Work	205
10.4	Closing Remarks	207
	Bibliography	208
A	PaTAS-TP Theorems and Proofs (Section 7.7)	219

B	All Plots for Chapter 7	225
B.1	Accuracy plots during NN Training	226
B.2	PaTAS Evaluation Results	228
C	More results from usecase chapter	249
C.1	PaTAS Confidence Signal: Per-Sample Input Trust Feedforward	249
C.2	Extended Coverage–Accuracy and ROC Analysis for Selective Prediction	250

List of Acronyms

AAU	Active Antenna Unit
ABF	Averaging Belief Fusion
Adam	Adaptive Moment Estimation
AI	Artificial Intelligence
ATL	Actual Trust Level
ATO	Atomic Trust Opinion
BBA	Basic Belief Assignment
BCF	Belief Constraint Fusion
BNN	Bayesian Neural Network
BNN+LV	Bayesian neural networks with latent variables
BPQ	Baseline-Prior Quantification
CBF	Cumulative Belief Fusion
CCF	Consensus & Compromise Fusion
C-ITS	Cooperative Intelligent Transportation Systems
CNN	Convolutional Neural Network
COMPAS	Correctional Offender Management Profiling for Alternative Sanctions
CUQ	Constant-Uncertainty Quantification
DAG	Directed Acyclic Graph
DenseNet	Dense Network
DSPG	Directed Series-Parallel Graph
DST	Dempster-Shafer Theory
ECE	Expected Calibration Error
EDL	Evidential Deep Learning
EWQ	Evidence-Weighted Quantification
GTSRB	German Traffic Sign Recognition Benchmark
GenIPTA	Inference Path Trust Assessment Generation

IAA	Inter-annotator agreement
ICT	Information and Communication Technology
IMA	Intersection Movement Assistance
IPTA	Inference Path Trust Assessment
KL	Kullback-Leibler
LLM	Large Language Model
LSTM	Long Short-Term Memory
MC dropout	Monte Carlo dropout
ML	Machine Learning
MLP	Multi-Layer Perceptron
NL	Nesting Level
NN	Neural Network
NNL	Node Nesting Level
OEM	Original Equipment Manufacturer
OOD	Out of Distribution
PaTAS	Parallel Trust Assessment System
PaTAS-TP	Parallel Trust Assessment System - Trust Propagation
PDF	Probability Density Function
PII	Personally Identifiable Information
PNPS	Parallel Non-intersecting Path Subnetwork
PPS	Parallel Path Subnetwork
ReLU	Rectified Linear Unit
ResNet	Residual Network
RQ	Research Question
RRU	Radio Resource Unit
SeqNN	Sequential Neural Network
SL	Subjective Logic
SLE	Subjective Logic Encodings
SP-graph	Series-Parallel Graph
STN	Subjective Trust Network
TAF	Trust Assessment Framework

TCP	Transmission Control Protocol
TLEE	Trust Level Evaluation Engine
TNA-SL	Trust Network Analysis with Subjective Logic
TNN	Trust Nodes Network
UQ	Uncertainty Quantification
V2V	vehicle-to-vehicle
WBF	Weighted Belief Fusion

List of Symbols and Notation

TABLE 1: List of Symbols and Notation

Symbol	Meaning / Description
Notational Conventions	
ω vs. T	ω denotes a general subjective opinion; T denotes an opinion specifically interpreted as <i>trust</i> .
$\mathbf{W}, \mathbf{b}$ vs. W, b	Bold symbols denote NN weight matrices / bias vectors; non-bold denotes the SL prior weight W and belief mass b_x respectively.
P (property vs. PNPS)	Two distinct uses, disambiguated by context: a trustworthiness property P in all chapters except 4, and a Parallel Non-intersecting Path Subnetwork (PNPS) in Ch. 4 only.
$\omega = (t, d, u)$ vs. (b, d, u, a)	The Actual Trust Level (ATL) is written as a triple $\omega = (t, d, u) = (b, d, u)$; the base rate a is left implicit (default $a = 0.5$ unless stated otherwise). The full binomial opinion retains a explicitly as (b, d, u, a) .
γ	Two distinct uses depending on context: the limit weights γ_X^A, γ_X^B in SL fusion operators vs. the normalization factor in Constant-Uncertainty Quantification. Always disambiguated by accompanying sub/superscripts or surrounding context.
Sets and Spaces	
Ω	Universe (frame of discernment)
$\mathbb{X}$	Opinion state space / domain
$\mathcal{R}(\mathbb{X})$	Hyperdomain of $\mathbb{X}$
Δ^{K-1}	$(K-1)$ -dimensional probability simplex
K	Cardinality of a domain (e.g. number of classes; in NN classification, equivalently the number of output neurons)
$\mathcal{D}$	Dataset
Probability and Evidence Theory	
$\Pr(\cdot)$	Classical / Bayesian probability
$Q(P)$	Second-order probability distribution over P
α, β	Beta distribution parameters (pseudo-counts of positive/negative evidence)
$\Gamma(\cdot)$	Gamma function
$\boldsymbol{\alpha} = (\alpha_1, \dots, \alpha_K)$	Dirichlet concentration vector
α_0	Sum of Dirichlet concentration parameters, $\sum_i \alpha_i$
$m(\cdot)$	Basic Belief Assignment (BBA), DST

Con	Conflict mass between two belief sources (Dempster's rule & BCF)
r, s	Positive/negative evidence counts supporting a proposition (shared meaning: Beta-opinion mapping)
W	Non-informative prior weight (Beta/Dirichlet $\leftrightarrow$ opinion mapping)

Subjective Opinions

$\omega_x = (b_x, d_x, u_x, a_x)$	Binomial opinion: belief, disbelief, uncertainty, base rate
$\omega_X = (\mathbf{b}, u, \mathbf{a})$	Multinomial opinion
ω_X^A	Opinion held by agent A about variable X
$\mathbb{P}[\omega]$	Projected probability of opinion ω (Eq. 2.2)

SL Operators

$(\cdot)$	Binomial opinion multiplication (logical conjunction, Def. 2.7)
$\odot$	Belief constraint fusion (BCF)
$\oplus$	Cumulative belief fusion (CBF)
$\overline{\oplus}$	Averaging belief fusion (ABF)
$\hat{\oplus}$	Weighted belief fusion (WBF)
$\odot\odot$	Consensus & compromise fusion (CCF)
$\ominus$	Inferential deduction operator
$\ominus$	Trust-revision operator
$\oslash$	Conservative trust-division operator
$\otimes_{TE}$	Two-edge path trust discounting
$\otimes_{ME}$	Multi-edge path trust discounting
$\otimes_{RE}$	Referral-edge path trust discounting

Graph Structure

v	Node (vertex)
e	Edge
$G = (V, E)$	Directed graph with vertex set V and edge set E
$\mathcal{N}(i)$	Set of incoming edges to neuron i
P	PNPS
$\mathcal{P}_G$	Set of all PNPSs of graph G
$NL_G(e)$	Nesting Level of edge e in G
$NNL_G(v)$	Node Nesting Level of v in G
$PNL_G(P)$	Nesting Level of PNPS P : $\min\{NL_G(e) \mid e \text{ is an edge of } P\}$

Dataset Trust Design Space

S	Lifecycle stage being assessed, $S \in \{\text{SAMPLING, COLLECTION, ANNOTATION, PROCESSING}\}$
G	Granularity at which a property is assessed, $G \in \{\text{DATASET, ENTRY, FEATURE}\}$

Dataset Trust-Source Quantification

m	Index/identifier of a trust-source metric
-----	---

q_m	Normalized quality score of metric m , $q_m \in [0, 1]$
τ_m	Tolerance threshold for lower-is-better metrics (Eq. 6.6)
A_m, L_m	Unacceptable / ideal bounds for higher-is-better metrics (Eq. 6.7)
K_m	Evidence-strength parameter of metric m (total evidence contributed, $r_m + s_m = K_m$)

Neural Network Notation

$\Theta = (\mathbf{W}, \mathbf{b}, \phi)$	Full NN parameterization
$\mathbf{W} = \{\mathbf{W}^{(l)}\}$	Weight matrices
$\mathbf{b} = \{\mathbf{b}^{(l)}\}$	Bias vectors
ϕ	Activation functions
L	Total number of layers
n_l	Number of neurons in layer l
$\mathcal{L}$	Loss function
lr	Learning rate
$\delta_i^{(l)}$	Error term of neuron i in layer l
$\hat{y}(x)$	Predicted class
N	Number of samples in the training dataset $\mathcal{D}$

Trust Opinion Quantification

w_x	Evidence weight in Evidence-Weighted Quantification (EWQ, Eq. 3.2)
U	Fixed target uncertainty level in Constant-Uncertainty Quantification (CUQ, Eq. 3.3)
γ	Normalization factor in CUQ, $\gamma = \frac{1 - U}{r_x + s_x}$ (Eq. 3.3)

Calibration

$\hat{p}(x)$	Predicted confidence for input x
M	Number of confidence bins
B_m	Set of sample indices whose predicted confidence falls in bin m
$\text{acc}(B_m)$	Empirical accuracy of samples in bin B_m
$\text{conf}(B_m)$	Average predicted confidence of samples in bin B_m
n	Total number of samples used for calibration evaluation*

Model and Parameters

l	Layer index
$n_i^{(l)}$	Neuron i in layer l
$\theta_{i,j}^{(l)}$	Model parameter connecting neuron j in layer $l-1$ to neuron i in layer l

Algorithm Inputs

g	Collection of gradients for all parameters in the batch
T_y	Trust opinion on label y
ϵ	Gradient sensitivity threshold in NODETRUST

Batch and Gradient Quantities

$T_{y_{\text{batch}}}$	Aggregated trust over labels in the current batch
$g_{i,j}^{(l)}$	Gradient of loss w.r.t. $\theta_{i,j}^{(l)}$
$g_i^{(l)}$	Gradient vector for neuron $n_i^{(l)}$
r	Count of weak gradients: $ g_{i,j}^{(l)} < \epsilon$
s	Count of strong gradients: $ g_{i,j}^{(l)} \geq \epsilon$

Trust Values

$T_{n_i y_{\text{batch}}}$	Trust in neuron n_i conditioned on batch labels
$T_{n_i \overline{y_{\text{batch}}}}$	Trust in neuron under incorrect labels (vacuous)
$T_{n_i Y_{\text{batch}}}$	Deduced trust in neuron n_i
$T_{\theta_{i,j}^{(l)}}$	Trust opinion on parameter $\theta_{i,j}^{(l)}$
$T_{x_j^{(l-1)}}$	Trust in the input to parameter $\theta_{i,j}^{(l)}$
T_D	Trust assessment of the training dataset D
T_x	Trust assessment of the input x
$T_{y'}$	Trust opinion on the model output / prediction y'
Pf_{Θ}	The parallel trust function: $Pf_{\Theta}(T_x) = T(f_{\Theta}(x))$

Chapter 7

Trust Propagation in Neural Network

The previous Chapter 5 presented the overall architecture of PaTAS, highlighting its two main components: (i) the assessment of trust in the data, and (ii) the propagation of trust through the model, from the training data to the learned parameters and from the input to the output via these parameters. While the trustworthiness of the data was addressed in the preceding chapter, this chapter focuses on the second component, namely the propagation of trust within Neural Network. In particular, it details how trust, initially derived from the dataset, is integrated into the model parameters during training and subsequently propagated from the input to the output during inference. This chapter formalizes the mechanisms that enable consistent and interpretable trust propagation across the different stages of the learning process, providing the operational core of the PaTAS-TP framework.

An interactive simulator demonstrating the trust propagation mechanisms described in this chapter is available at https://ouatt-isma.github.io/patas_simulator.

Connection with the preceding chapters. This chapter builds directly on two prior contributions of the thesis: the Trust Network Analysis with Subjective Logic foundations introduced in Chapter 2, which define how the Subjective Logic (SL) operators are used to combine trust opinions over Directed Series-Parallel Graphs, and the dataset-trust quantification methodology developed in Chapter 6, which produces the feature-, instance-, and label-level trust opinions consumed here as inputs ($T(x)$, $T(y)$). The present chapter closes the loop by specifying how these *dataset-derived opinions* are transformed into *parameter trust* during training and *input trust* into *output trust* during inference, thereby answering Research Question (RQ) 3.

7.1 Introduction

The preceding chapter addressed the first component of PaTAS: the quantification of trustworthiness in training datasets. It established that trust in a dataset can be assessed at multiple granularities, from global distributional properties to individual feature-level reliability, and provided formal tools for computing SL opinions over data quality. However, quantifying dataset trustworthiness is only a prerequisite. The central challenge, as formulated in RQ 3, is to propagate this trust through the learning and inference processes, such that the reliability of a prediction can be systematically traced back to the reliability of both the data and the parameters from which it originates.

As discussed in Chapter 2, the trustworthiness of a Neural Network output cannot be treated as a static property of the model alone. During training, model parameters are updated through gradient-based optimization driven by labeled data. If this data is noisy, biased, or corrupted, the resulting parameter updates inherit these deficiencies, embedding unreliability directly into the learned model. During inference, inputs are transformed through successive layers parameterized by these learned weights; consequently, unreliability originating from either the input or the parameters may propagate and accumulate throughout the network, ultimately affecting the final prediction. Standard performance metrics such as accuracy fail to capture this phenomenon, as they conflate correctness with reliability. A model may achieve high accuracy on a poisoned or biased dataset while producing predictions that remain fundamentally untrustworthy.

Existing approaches to trust and uncertainty propagation in NNs, reviewed in Section 2.3, address aspects of this problem but leave important gaps. Uncertainty propagation methods, such as those of Monchot et al. [10] and Astudillo and Neto [11], model the transmission of statistical variability through network layers, but treat uncertainty primarily as a distributional property of the input. As such, they do not capture the joint influence of dataset reliability and parameter trustworthiness on the final prediction. The DeepTrust framework of Cheng et al. [12] is more closely related, as it propagates dataset-level trust opinions through a NN to derive a global model trust estimate. However, as discussed in Section 2.3.3, DeepTrust’s formulation leaves several theoretical questions open. Its mapping of NN addition to SL fusion and multiplication to SL opinion multiplication mixes operators from distinct algebraic domains, since fusion operates over agents’ opinions while multiplication operates over variables, without a principled justification for this correspondence. Furthermore, its trust computation is specified only at the output layer, and the mechanism does not admit a straightforward extension to intermediate layers of deep architectures. As a result, while DeepTrust motivates the use of SL for trust assessment in NNs, the semantics of its propagation in deeper settings remain underspecified.

This chapter provides a formal and operational answer to RQ 3 by introducing a principled framework for trust propagation in NNs. Building on the foundations of SL and the optimized trust inference mechanisms introduced in Chapter 4, we model trust as a transitive, structure-aware, and evidence-driven quantity that propagates through the network in parallel with standard NN computation. The proposed framework integrates two complementary sources of trust: (i) the reliability of input features at inference time, and (ii) the reliability of model parameters, which themselves are derived from the trustworthiness of the training dataset. Importantly, this propagation is performed without modifying the underlying NN, thereby preserving compatibility with existing architectures. Methodologically, PaTAS-TP is a *white-box* framework: it exploits the internal structure of the NN, namely its layer topology, parameters, and activation paths, to compute trust at each intermediate computation rather than treating the model as an opaque input-output mapping. This places PaTAS-TP in direct contrast with the black-box calibration approach of Chapter 8.

More specifically, this chapter introduces the following contributions. First, we define a parallel trust computation mechanism based on Trust Nodes and Trust Functions, which mirror the structure of NN computations and enable consistent trust propagation using SL operators such as fusion and discounting. Second, we propose a parameter-trust update mechanism that estimates the reliability of learned parameters during training by leveraging gradient information, features trust, and

label trust, thereby aligning parameter trust with the learning dynamics; the gradient-stability threshold it relies on is given a scale-tied, task-relative formulation that removes its only tuning burden. Third, we introduce the Inference Path Trust Assessment (IPTA), a context-aware mechanism that computes instance-specific trust scores by exploiting the activation paths involved in the prediction. ~~—~~, and generalize it to convolutional architectures through activity-weighted path conditioning. Fourth, we instantiate the input-trust route with a conformity-based construction that derives per-feature opinions from training-data statistics, and compose it with the path trust into a single per-sample score. Finally, we demonstrate through empirical evaluation, including a comparative battery against established uncertainty and detection baselines and a test-data-free training-quality audit, that the proposed framework produces interpretable, consistent, and convergent trust estimates that reveal reliability gaps not captured by standard performance metrics.

The remainder of this chapter is organized as follows. Section 7.2 introduces the formal foundations of trust-aware NN computation and defines the parallel trust function. Section 7.3 presents the PaTAS-TP architecture, decomposing it into four functional modules: Trust Feedforward, Output-Trust Aggregation, Inference Path Trust Assessment Generation (GenIPTA), and Parameter-Trust Update. Section 7.4 illustrates the core mechanisms of the Trust Feedforward through a simple perceptron example. Section 7.5 generalizes these mechanisms to deep NNs by introducing the Trust Function and layer-wise propagation, and discusses how Directed Series-Parallel Graph (DSPG) structure ensures the soundness of trust operators in non-sequential architectures. Section 7.6 details the Parameter-Trust Update mechanism that refines trust in learned parameters during training. Section 7.7 establishes theoretical properties of the framework, including convergence guarantees. Section 7.8 instantiates the per-sample trust assessment used at inference time: the conformity-based input-trust construction, its a-priori applicability diagnostics, and the composed trust score. Section 7.9 provides comprehensive experimental validation ~~on three datasets~~ of increasing complexity, from controlled degradations on three datasets to a comparative evaluation against uncertainty and detection baselines. Finally, Section 7.10 discusses key findings and practical implications, and Section 7.11 concludes the chapter.

7.2 Foundations of Trust Propagation in Neural Networks

This section introduces the formal structures needed to represent and propagate trust alongside standard NN computation, namely during feedforward and back-propagation.

7.2.1 Trust Representation in Neural Network

As established previously, a Sequential Neural Network f_{Θ} with parameters $\Theta = (\mathbf{W}, \mathbf{b}, \phi)$ defines a structured computational graph that maps an input x to an output y' through a sequence of layer-wise transformations:

$$y' = f_{\Theta}(x).$$

Rather than reintroducing the full formulation, we emphasize that this computation induces a Directed Acyclic Graph (DAG) in which information flows from input features to output predictions through intermediate activations and parameters. Trust propagation inherits the compositional structure of this computation: just

as numerical values flow through the network from inputs to outputs via intermediate activations and parameters, trust assessments can be propagated along the same directed dependencies.

To enable trust-aware reasoning, we associate trust opinions with the key components of the NN. Specifically, we consider:

- **Input Trust:** A trust assessment $T(x)$ assigned to the input x , defined at feature-level granularity as introduced in Chapter 6.
- **Label Trust:** A trust assessment $T(y)$ assigned to the label y associated with a training sample $(x, y) \in \mathcal{D}$, reflecting the reliability of the annotation process. It serves as a conditioning factor in the calculation of trust in the learnable parameters during training.
- **Parameter Trust:** A trust assessment $T(\theta)$ for each model parameter $\theta \in \Theta$, reflecting the reliability of learned parameters as a function of the training data and learning process.
- **Output Trust:** A trust assessment $T(y')$ associated with the model output $y' = f_{\Theta}(x)$.

The central problem addressed in this chapter is therefore twofold. During training, trust must flow from the dataset, specifically from the reliability of input features and labels, into the learned model parameters, so that data quality is reflected in the resulting weights. During inference, trust must then be carried forward from the input features and the trust parameters to the model output, following the same computational dependencies that govern the forward pass. In both cases, propagation is understood as a transitive relationship: the trustworthiness of a downstream component is conditioned on, and inherits from, the trustworthiness of the upstream components on which it depends.

7.2.2 Parallel Trust Function

To address this problem, we introduce the notion of a *parallel trust function*, which mirrors the structure of the NN.

Definition 7.1 (Parallel Trust Function). Let f_{Θ} represent NN with parameters Θ , and let $T(x)$ denote trust assessments over the input of the NN. The *parallel trust function*, denoted Pf_{Θ} , is defined as:

$$Pf_{\Theta}(T(x)) = T(f_{\Theta}(x)),$$

where $T(f_{\Theta}(x))$ is the resulting trust opinion on the output of the NN, reflecting both the trust assigned to the input features and the trust encoded in the learned parameters Θ .

The parallel trust function is constructed to satisfy two key principles:

- **Structural Consistency:** The trust propagation mechanism mirrors the computational structure of the NN. Each transformation in f_{Θ} has a corresponding transformation in Pf_{Θ} . This mirroring ensures that trust propagation is sensitive to the architecture of the model: a change in network topology, such as the addition of a layer, is automatically reflected in the trust propagation from input to output.

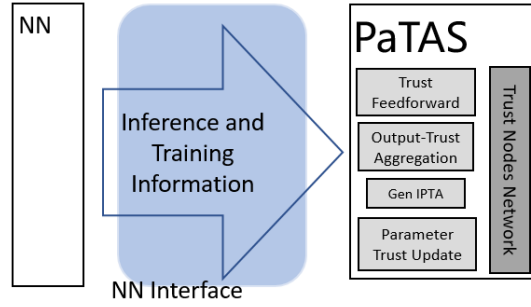


FIGURE 7.1: High-Level Overview of the Trust Propagation in NN

- **Evidence Integration:** Trust is computed as a function of both input reliability ($T(x)$) and parameter reliability (encoded in Pf_{Θ}), capturing their joint influence on the output. This joint treatment is essential, since high-quality inputs processed through poorly trained parameters may still yield unreliable predictions, and conversely, well-trained parameters may fail to compensate for fundamentally untrustworthy inputs.

Conceptually, Pf_{Θ} defines a second computational graph operating over trust opinions rather than numerical values. While the NN computes predictions, the parallel trust function computes their associated trustworthiness. Both processes share the same structural dependencies but operate in different domains. This formulation enables trust to be modeled as a *structure-aware and evidence-driven quantity* that propagates through the network. In the following section, we present the overall architecture of PaTAS-TP, which operationalizes this abstraction as a concrete system.

7.3 PaTAS-TP: Architecture Overview

To extend the formal foundations established in the preceding section, this section presents the architecture of PaTAS-TP, the concrete system through which the parallel trust function is created and operationalized. As illustrated in Fig. 7.1, this parallel system is organized around a central abstraction called the Trust Nodes Network, and is decomposed into four functional modules: Trust Feedforward, Output-Trust Aggregation, GenIPTA, and Parameter-Trust Update. Each module corresponds to a distinct phase of the NN lifecycle, namely training feedforward, training backpropagation, and inference.

The backbone of PaTAS-TP is the *Trust Nodes Network (TNN)*, which mirrors the topology of the underlying NN at the level of trust opinions. The TNN is built from a single primitive: the *Trust Node*, which is the trust-domain counterpart of a neuron.

Definition 7.2 (Trust Node). A *Trust Node* is an abstract computational unit associated with a neuron in a NN. It receives trust opinions on the neuron’s inputs and produces a trust opinion on the neuron’s output. The structure of a Trust Node mirrors that of its corresponding neuron, but its computation is defined over trust opinions.

The parameters of the Trust Nodes are trust opinions rather than real-valued weights; they are initialized as vacuous opinions and refined during training by the Parameter-Trust Update module. The TNN is a structured composition of Trust Nodes that mirrors the topology of the underlying NN, propagating trust assessments layer by

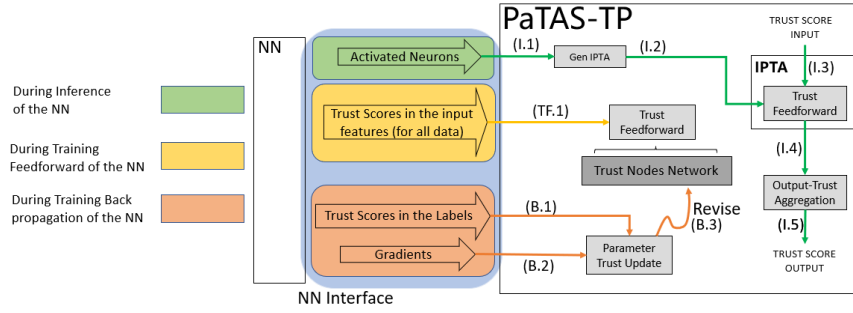


FIGURE 7.2: Functional flow of PaTAS-TP integrated with a NN, showing the interaction between the Trust Feedforward (TF), Parameter-Trust Update (B), and GenIPTA/Output-Trust Aggregation (I) modules across the training and inference phases. **Feedforward (Training Phase):** The NN interface extracts trust scores from input features for all data being processed. These trust scores are passed to the *Trust Feedforward* module (TF.1), which propagates them through the *TNN* in parallel with the NN computations. During training, *Trust Feedforward* stores the trust scores of all intermediate computations in the NN, to be reused in backpropagation. **Backpropagation (Training Phase):** Once the NN computes gradients, the trust scores of the labels (B.1) and the gradients (B.2) are provided to the *Parameter-Trust Update*. Gradients indicate how dataset labels affect parameter updates, while label trust determines whether these updates should be considered reliable. The stored intermediate trust values are also incorporated in this process. Finally, the *Parameter-Trust Update* module refines the trust values of the *TNN* parameters (B.3), aligning them with the evolving learning dynamics. **Inference Phase:** During operation, contextual information such as the set of activated neurons is retrieved. This information is passed to *GenIPTA* (I.1), which constructs an *IPTA* corresponding to the specific inference path (I.2). The *IPTA* takes input trust scores (I.3), propagates them along the path, and produces trust scores for each NN output. These can then be passed to the *Output-Trust Aggregation* (I.4) which will combine them into a single consolidated trust score representing the reliability of the prediction (I.5).

layer using the trust operations defined over each node. Each Trust Node receives trust opinions from its predecessors, applies its corresponding transformation, and passes the resulting opinion to its successors.

The TNN serves as the shared underlying representation for all four modules of PaTAS-TP.

Trust Feedforward The Trust Feedforward function propagates trust values through the TNN in alignment with the NN’s inference flow. It mirrors the layer-wise computations of the original model, but operates entirely on trust values. At each layer, trust in the inputs and parameters is combined using the trust operations, capturing how evidence flows through the network. This process produces a trust opinion for each output neuron of the NN while also storing intermediate trust scores, which are later used by the Parameter-Trust Update.

Output-Trust Aggregation The Trust Feedforward function produces a vector of trust values, one for each output neuron. The goal of Output-Trust Aggregation is to combine these individual opinions into a single, consolidated trust score representing the overall trust in the network’s output. This combination can be performed using a SL fusion operator, which fuses the trust opinions of all output Trust Nodes into one aggregated opinion. Alternatively, instead of aggregating across all outputs, one may directly consider the trust assigned to the final decision. For instance, in digit classification, if the model outputs a probability 0.9 for class ‘1’, the trust in the decision can be taken as the trust score associated with the output neuron for class ‘1’. More generally, the aggregation can be weighted by the strength of each output, for instance by the softmax probabilities, such that higher-confidence outputs contribute more heavily to the consolidated trust score. Finally, because every layer applies one round of discounting, aggregated trust decays geometrically with depth; a depth-normalized variant therefore rescales the aggregated opinion by the inverse of this per-layer decay, making trust values comparable across architectures of different depths. Both the raw and the depth-normalized aggregate are used in the evaluation.

Inference Path Trust Assessment Generation (GenIPTA) The purpose of the GenIPTA module is to dynamically construct a function tailored to a single, specific inference. This function, called IPTA, reflects how trustworthy the exact computational path taken during that inference is. When an inference is performed, contextual information is recorded, such as the list of neurons activated along the path. In this case, the activation trace is used to instantiate a temporary subnetwork of Trust Nodes containing only the Trust Nodes corresponding to those activations, thereby mirroring the precise inference path of the NN. Contextual information is not limited to activation traces. Typically, all neurons are activated to some degree, but only a subset of these activations is strongly relevant to the decision. GenIPTA can therefore operate on truncated activation sets that retain only the most relevant neurons, which allows for more focused and interpretable trust assessments of the actual decision-making process. The GenIPTA can also work in a more complex way by using the actual activation values of all neurons, performing a weighted trust assessment where stronger activations contribute more heavily to the trust inference.

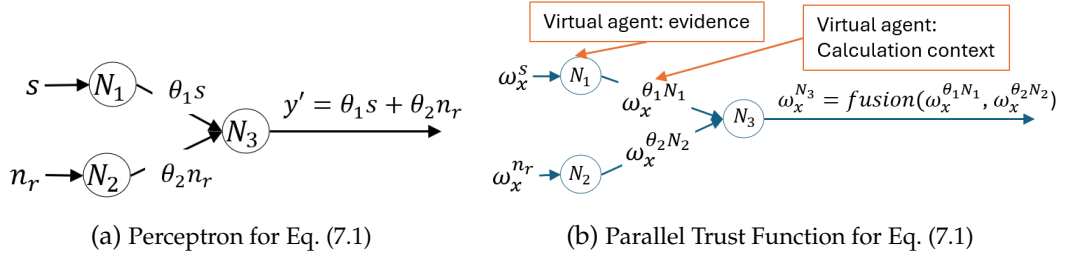


FIGURE 7.3: Trust-propagation based solely on input-feature trust

Parameter-Trust Update The Parameter-Trust Update refines the trust opinions stored in the TNN parameters during training, based on gradient information, input trust, and label trust to align parameter reliability with the evolving learning dynamics; its full formulation is presented in Section 7.6.

The detailed functional flow of PaTAS-TP across feedforward, backpropagation, and inference is illustrated in Fig. 7.2.

With the architecture of PaTAS-TP established, the following section introduces the perceptron as a minimal working example, providing concrete intuition for the operators mechanisms that underpin the Trust Feedforward computation before their generalization to deep architectures in Section 7.5.

7.4 Trust Feedforward in a Simple Model: The Perceptron Case

To illustrate the proposed trust propagation framework for the feedforward, we begin with the simple case of a single-layer perceptron. This setting provides an intuitive and tractable example that highlights the key mechanisms underlying the parallel trust function before extending them to deep NNs.

Consider a perceptron defined by a linear function:

$$y' = \theta_1 s + \theta_2 n_r, \quad (7.1)$$

where s and n_r denote input features (e.g., size and number of rooms in a apartment price estimation problem), and θ_1, θ_2 are the corresponding parameters.

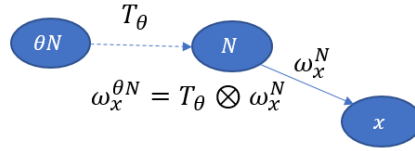
We assume that trust assessments are available for:

- the input features: T_s and T_{n_r} ,
- the parameters: T_{θ_1} and T_{θ_2} .

The objective is to determine the resulting trust opinion $T(y')$ on the output, given these inputs.

Trust Propagation from Inputs We first consider the simplified case where the parameters are assumed to be fully trusted. In this setting, the output is entirely determined by the input features, and trust propagation reduces to combining the trust opinions associated with each feature. The model specified in Eq. (7.1) is equivalent to the NN depicted in Fig. 7.3a. This network employs a linear transformation without an activation function. The input vector is:

$$x = \begin{pmatrix} s \\ n_r \end{pmatrix}$$

FIGURE 7.4: Trust-propagation Trust Network with parameter trust¹.

Let ω_x^s be the trust opinion on x based on s and $\omega_x^{n_r}$ the trust opinion on x based on n_r . Since the output neuron N_3 computes Eq. (7.1), we associate this computation with two agents: $\theta_1 N_1$ from the context of computing $\theta_1 \cdot s$ and $\theta_2 N_2$ from the context of computing $\theta_2 \cdot n_r$. The trust opinion of the output neuron N_3 on x is then: $\omega_x^{N_3} = \text{fusion}(\omega_x^{\theta_1 N_1}, \omega_x^{\theta_2 N_2})$.

The choice of the fusion operator (that we will later denote by $\oplus$) depends on the semantics of s and n_r (see Section 2.2.3). In this example, since s and n_r represent independent evidence, the fusion operator to use is cumulative fusion.

Assuming full trust in the parameters θ_1 and θ_2 , we have:

$$\begin{aligned}\omega_x^{\theta_1 N_1} &= \omega_x^{N_1} = \omega_x^s = T_s \quad (\text{Trust in the feature } s \text{ of } x) \\ \omega_x^{\theta_2 N_2} &= \omega_x^{N_2} = \omega_x^{n_r} = T_{n_r} \quad (\text{Trust in the feature } n_r \text{ of } x)\end{aligned}$$

Since the output value is deterministically related to the input via

$$y' = f(x),$$

the trust opinion of N_3 on y' is the same as the trust opinion already computed for x . Clearly, The transformation performed by f is encoded in the agent algebra, not in the variable algebra. Thus,

$$\begin{aligned}\omega_{y'}^{N_3} &= \omega_x^{N_3} = \omega_x^{\theta_1 N_1} \oplus \omega_x^{\theta_2 N_2} \\ &= \omega_x^s \oplus \omega_x^{n_r} = T_s \oplus T_{n_r}\end{aligned}\tag{7.2}$$

In summary, if we define $T_x = (T_s \quad T_{n_r})$, then:

$$Pf_{\Theta}(T_x) = Pf_{\Theta}((T_s \quad T_{n_r})) = T_s \oplus T_{n_r}$$

This formulation reflects the fact that the output trust aggregates evidence from multiple input sources, each contributing to the final prediction.

Incorporating Parameter Trust We now consider the general case where parameters are not assumed to be fully reliable. In this setting, parameter trust acts as a filtering mechanism that modulates the contribution of each input feature. Unlike the previous problem, where we fully trusted θ_1 and θ_2 , here we do not fully trust these parameters, and we must account for their trust assessments. For the network to incorporate the trust in θ_1 and θ_2 , we adjust the trust opinions on the features accordingly. The trust opinions are now refined. As depicted in Fig. 7.4, we model this as a small Subjective Trust Network. therefore:

$$\omega_x^{\theta_1 N_1} = T_{\theta_1} \otimes \omega_x^{N_1} \text{ and } \omega_x^{\theta_2 N_2} = T_{\theta_2} \otimes \omega_x^{N_2}\tag{7.3}$$

where $\otimes$ is a trust discounting operator. This results is consistent with the solution for problem 1 as for fully trusted $T_{\theta} = (1, 0, 0)$, we have $\omega_x^{\theta N} = T_{\theta} \otimes \omega_x^N = \omega_x^N$ (for

any neuron N).

Finally, the resulting output trust $\omega_{y'}^{N_3}$ is calculated by the fusion of these adjusted trust opinions:

$$Pf_{\Theta}(T_x) = Pf_{\Theta}((T_s \quad T_{n_r})) = (T_{\theta_1} \otimes T_s) \oplus (T_{\theta_2} \otimes T_{n_r}) \quad (7.4)$$

Thus, we adjust the trust in the network output based on the trust in both the parameters and the features, ensuring that the trust propagation takes into account the trust in the parameters.

This expression highlights two key mechanisms:

- **Trust Discounting:** Parameter trust modulates the reliability of each input contribution.
- **Trust Fusion:** The resulting contributions are aggregated to form the final output trust.

The perceptron case reveals the fundamental principles of trust propagation in NNs. First, trust is *compositional*: the output trust is constructed from the contributions of individual components. Second, trust is *transformed*: parameter trust acts as a filter that attenuates or amplifies the influence of input trust. Third, trust is *aggregated*: multiple sources of evidence are combined using fusion operators.

In the next section, we generalize these principles to deep NNs, where trust must be propagated across multiple layers and intermediate activations.

7.5 Generalization to Deep NNs

The perceptron example introduced in the previous section highlights the fundamental mechanisms of trust propagation during feedforward. We now extend these principles to deep NNs, where trust must be propagated across multiple layers and intermediate representations. Consider a NN (Θ) composed of L layers, where each layer defines a transformation:

$$h^{(l)} = f^{(l)}(W^{(l)}h^{(l-1)} + b^{(l)}), \quad l = 1, \dots, L,$$

with $h^{(0)} = x$ and $h^{(L)} = y'$.

In the trust domain, we associate a trust opinion $T^{(l)}$ to each layer output $h^{(l)}$, such that:

$$T^{(l)} = T(h^{(l)}), \quad l = 0, \dots, L,$$

In particular

$$T^{(0)} = T(h^{(0)}) = T_x, \quad T^{(L)} = T(h^{(L)}) = T(y') = Pf_{\Theta}(T_x)$$

Trust propagation is then defined recursively as:

$$T^{(l)} = f_T^{(l)}(T^{(l-1)}, T_{\Theta}^{(l)}),$$

where $T_{\Theta}^{(l)}$ denotes the trust associated with the parameters of layer l , and $f_T^{(l)}$ is the trust transformation corresponding to the NN operation at that layer. This formulation extends the parallel trust function introduced in Section 7.2 to a layer-wise decomposition, where trust is propagated sequentially through the network in a

manner consistent with the forward computation. In practice, as developed in the GenIPTA module, the parallel function is not defined once for the whole network but instantiated per inference, since the active computational path depends on the specific input.

To formalize this process, we introduce the concept of trust function.

Definition 7.3 (Trust Function). A *Trust Function* models the transformation of trust through a Trust Node. It defines how trust opinions on the inputs of a neuron are combined to produce a trust opinion on the output. For a neuron that computes $z^{(l)} = \sum \theta_i^{(l)} \cdot x_i^{(l)}$, $x^{(l+1)} = \phi^{(l)}(z^{(l)})$, the corresponding trust computation is given by:

$$T_{z^{(l)}} = \bigoplus_{i=1}^d T_{x_i^{(l)}} \otimes T_{\theta_i^{(l)}}, \quad T_{x^{(l+1)}} = T_{\phi^{(l)}}(T_{z^{(l)}}),$$

- $\otimes$ is a trust discounting operator,
- $\oplus$ is a trust fusion operator,
- $T_{\phi}^{(l)}$ is the trust-equivalent of the activation function. In this work, we set it to identity function as we use ReLU as activation function.

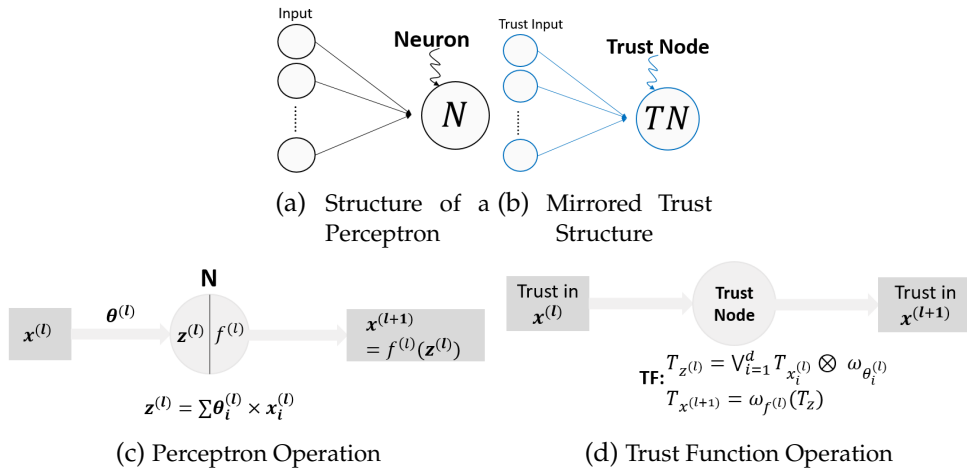


FIGURE 7.5: Illustration of the Trust Node structure and computation

Fig. 7.5 illustrates the structural correspondence between a standard neuron and its associated Trust Node. Figs. 7.5a and 7.5b show how the mirrored trust structure is derived from the original perceptron, while Figs. 7.5c and 7.5d contrast the numerical computation of the neuron with the trust-domain computation of the Trust Function, in which multiplication is replaced by discounting and summation by fusion.

7.5.1 Extension to Convolutional Architectures

The construction above is stated for fully connected layers, but nothing in it is specific to them; what changes in a Convolutional Neural Network (CNN) is weight sharing. PaTAS-TP mirrors a convolutional layer with c_{in} input channels, c_{out} output channels, and kernel size k by one opinion matrix of shape $(c_{in}k^2 + 1) \times c_{out}$, exactly the flattened kernel matrix that also carries the layer's gradients, so the Parameter-Trust Update of Section 7.6 applies unchanged. For the feedforward, the input pixel opinions

Algorithm 5 Trust Feedforward

Require: Input trust $T^{(0)} = T(x)$, parameter trust $\{T_{\Theta}^{(l)}\}_{l=1}^L$

Ensure: Output trust $T(y') = T^{(L)}$

- 1: **for** each layer $l = 1, \dots, L$ **do**
- 2: **for** each neuron i in layer l **do**
- 3: Compute discounted contributions:

$$T_{\text{contrib},j} = T_{\theta_{ij}^{(l)}} \otimes T_{x_j}^{(l-1)}, \quad \forall j \in \mathcal{N}(i)$$

- 4: Aggregate contributions via fusion:

$$T_{z_i}^{(l)} = \bigoplus_{j \in \mathcal{N}(i)} T_{\text{contrib},j}$$

- 5: Apply activation trust:

$$T_{x_i}^{(l)} = T_{\phi^{(l)}}(T_{z_i}^{(l)})$$

- 6: **end for**
- 7: **end for**
- 8: **return** $T(y') = T^{(L)}$

are fused into per-channel opinions and the layer is applied as the standard dense discount-fusion rule on the flattened kernel matrix; this *uniform-trust reduction* is exact whenever the input trust is spatially uniform (the trusted, vacuous, and distrusted profiles used throughout) and an approximation otherwise. ReLU and pooling remain identity in opinion space, and a residual skip connection is mapped to averaging fusion of the block's input and output opinions.

The IPTA generalizes as well, with one conceptual change. In a fully connected network an inference path selects *weights*: inactive neurons remove their rows from the trust computation. Under weight sharing a path cannot select weights, only *applications* of a filter, since the same kernel is applied at every spatial position. The convolutional IPTA therefore weights each channel's contribution by its *participation degree*: the fraction of spatial positions at which the channel's post-ReLU output is positive for the given input. Formally, the discount-fusion step becomes a weighted fusion in which row j of the opinion matrix enters with weight w_j equal to the producing channel's participation (bias rows keep weight 1). For binary participation this reduces *exactly* to the row-pruned computation of the fully connected IPTA, so the convolutional variant is a strict generalization: channels the forward pass never activated contribute nothing, and partially active channels contribute in proportion to their actual involvement in the decision.

Alg. 5 formalizes this layer-wise computation as the complete Trust Feedforward pipeline. This generalization demonstrates that trust propagation can be integrated into deep NNs without altering their structure, by defining a parallel computation over trust opinions. It makes the link between output trust $T_{y'}$, input trust T_x , and parameter trust T_{θ} :

$$T_x \xrightarrow{T_{\theta}} T_{y'}.$$

In the next section, we focus on how parameter trust is computed during training, completing the link between dataset trust and model trustworthiness:

$$T_{\mathcal{D}} \rightarrow T_{\theta}.$$

7.6 Parameter-Trust Update

The Parameter-Trust Update is the mechanism through which PaTAS-TP adapts the trust associated with model parameters during training. Its objective is to ensure that parameter trust reflects both the observed behavior of the model and the reliability of the training data. Its design is inspired by the standard backpropagation algorithm used in NN training, where gradients quantify how much each parameter contributes to the output error. By leveraging these gradients as evidence, PaTAS-TP updates parameter trust in a way that mirrors the learning dynamics of the underlying model.

To ground the design, we briefly recall the standard parameter update in NN training. For a parameter θ , gradient descent updates are given by:

$$\theta \leftarrow \theta - l_r \frac{\partial \mathcal{L}}{\partial \theta}, \quad (7.5)$$

where l_r is the learning rate, $\mathcal{L}$ is the loss function, and $\frac{\partial \mathcal{L}}{\partial \theta}$ is the gradient. These gradients are computed recursively using the chain rule:

$$\frac{\partial \mathcal{L}}{\partial \theta_{i,j}^{(l)}} = \delta_i^{(l)} x_j^{(l-1)}, \quad (7.6)$$

where $\theta_{i,j}^{(l)}$ denotes the weight connecting neuron j in layer $(l-1)$ to neuron i in layer l , $\delta_i^{(l)}$ is the error term of neuron i , and $x_j^{(l-1)}$ is the activation from the previous layer.

In particular, Eq. (7.6) shows that gradients factor into an error term and an input activation, revealing that parameter updates depend jointly on gradient signals, input activations, and learning dynamics. PaTAS-TP mirrors this structure by incorporating gradient information, input trust, and label trust as analogous sources of evidence in the Parameter-Trust Update. At a high level, the parameter-trust update process combines three sources of information: (i) gradient-based evidence derived from backpropagation, (ii) trust in the labels within the current training batch, and (iii) trust in the input features and intermediate computations recorded during feed-forward. These elements are integrated to revise and refine the trust assigned to each parameter. The notation used throughout this subsection is summarized in Table 1, and the complete update procedure is given in Alg. 6.

The procedure described in Alg. 6 operates at the level of mini-batches and iterates over all layers and neurons of the network. For each neuron, it first derives a trust assessment based on gradient evidence and the reliability of the training labels, and then propagates this information to update the trust associated with its incoming parameters. More specifically, the update process can be decomposed into three main stages: (i) aggregation of label trust at the batch level, (ii) estimation of neuron-level trust using gradient information, and (iii) revision and adjustment of parameter trust using both inferred neuron trust and auxiliary factors such as input trust. We now describe these steps in detail.

Algorithm 6 Parameter-Trust Update Algorithm

```

1: function PARAMETERTRUSTUPDATE( $g, T_y, \epsilon$ )
2:   Summary: Revises and updates the trust parameters of the Trust Nodes by
   combining gradient evidence, label trust, and neuron input trust, under mini-
   batch training.
3:   Step 1: Compute the aggregated trust in the labels
4:    $T_{y_{\text{batch}}} \leftarrow \bigwedge_{y \in y_{\text{batch}}} T_y$ 
5:   for each layer  $l$  do
6:     for each neuron  $n_i^{(l)}$  do
7:       Step 2: Gather gradient  $g$  for neuron  $n_i^{(l)}$ 
8:        $g_i^{(l)} \leftarrow \{g_{i,j}^{(l)} \mid j \in \mathcal{N}(i)\}$ 
9:       Step 3: Compute trust for neuron  $n_i^{(l)}$ 
10:       $T_{n_i|y_{\text{batch}}} \leftarrow \text{NODETRUST}(g_i^{(l)}, T_{y_{\text{batch}}}, \epsilon)$ 
11:      Step 4: Deduce overall trust in neuron  $n_i^{(l)}$ 
12:       $T_{n_i||Y_{\text{batch}}} \leftarrow \text{DEDUCETRUST}(T_{n_i|y_{\text{batch}}}, T_{n_i|\overline{y_{\text{batch}}}}, T_{y_{\text{batch}}})$ 
13:      for each incoming edge  $j$  to  $n_i^{(l)}$  do
14:        Step 5: Revise parameter trust with node trust
15:         $T_{\theta_{i,j}^{(l)}} \leftarrow \text{REVISE}(T_{\theta_{i,j}^{(l)}}, T_{n_i||Y_{\text{batch}}})$ 
16:        Step 6: Update parameter trust with auxiliary factors
17:         $T_{\theta_{i,j}^{(l)}} \leftarrow \text{UPDATE}(T_{\theta_{i,j}^{(l)}}, T_{x_j^{(l-1)}}, T_{y_{\text{batch}}})$ 
18:      end for
19:    end for
20:  end for
21: end function
22: function NODETRUST( $g_i^{(l)}, T_{y_{\text{batch}}}, \epsilon$ )
23:   Count  $r$ : #edges with  $|g_{i,j}^{(l)}| < \epsilon$ 
24:   Count  $s$ : #edges with  $|g_{i,j}^{(l)}| \geq \epsilon$ 
25:   Map  $(r, s)$  into a binomial opinion using Baseline-Prior Quantification
26:   return  $T_{n_i|y_{\text{batch}}}$ 
27: end function
28: function DEDUCETRUST( $T_{n_i|y_{\text{batch}}}, T_{n_i|\overline{y_{\text{batch}}}}, T_{y_{\text{batch}}}$ )
29:    $T_{n_i|\overline{y_{\text{batch}}}} \leftarrow (0, 0, 1)$ 
30:   return  $T_{n_i||Y_{\text{batch}}} = T_{y_{\text{batch}}} \odot (T_{n_i|y_{\text{batch}}}, T_{n_i|\overline{y_{\text{batch}}}})$ 
31: end function
32: function REVISE( $T_{\theta_{i,j}^{(l)}}, T_{n_i||Y_{\text{batch}}}$ )
33:   return  $T_{\theta_{i,j}^{(l)}} \odot T_{n_i||Y_{\text{batch}}}$ 
34: end function

```

The algorithm runs once per batch and begins by computing a combined trust opinion over all labels in the current batch (Line 3): $T_{y_{\text{batch}}} = \bigwedge_{y \in y_{\text{batch}}} T_y$.

This combined opinion serves as a foundation for conditioning the trust in each parameter.

For each neuron $n_i^{(l)}$ at index i in layer l , we compute:

- $T_{n_i^{(l)}|y_{\text{batch}}}$ (Line 9), the trust conditioned on the current batch labels. This is inferred by checking each incoming weight gradient $g_{i,j}^{(l)}$: if $|g_{i,j}^{(l)}| < \epsilon^2$ it is counted as positive evidence r , otherwise as negative evidence s . These counts are then mapped into a binomial opinion using the Baseline-Prior Quantification model.
- $T_{n_i^{(l)}|\overline{y_{\text{batch}}}}$, the trust when the true batch labels are not y_{batch} . Since no concrete evidence is available for this case, it is initialized to a vacuous opinion: $(0, 0, 1)$.
- A deduced trust $T_{n_i^{(l)}||y_{\text{batch}}}$ (Line 11) using the inferential deduction operator $\odot$, combining the two conditional opinions with $T_{y_{\text{batch}}}$.

Then for each incoming edge j of neuron i , the parameter trust $T_{\theta_{i,j}^{(l)}}$ is updated in two stages:

1. Revision with deduced trust using fusion (Line 14):

$$T_{\theta_{i,j}^{(l)}} \leftarrow T_{\theta_{i,j}^{(l)}} \odot T_{n_i^{(l)}||y_{\text{batch}}}$$

2. Adjustment with auxiliary factors (Line 16): During training, the update of parameter $\theta_{i,j}^{(l)}$ depends not only on its current value but also on auxiliary factors such as the input $x_j^{(l-1)}$, and the label y_{batch} (see Eqs. (7.5) and (7.6)). To reflect this, we adjust parameter trust using:

$$T_{\theta_{i,j}^{(l)}} \leftarrow T_{\theta_{i,j}^{(l)}} \odot (T_{x_j^{(l-1)}} \oslash T_{y_{\text{batch}}}).$$

Here, $\odot$ is the binomial multiplication operator and $\oslash$ is defined as:

$$\begin{aligned} (b, d, u) &= (b_1, d_1, u_1) \oslash (b_2, d_2, u_2), \\ b &= \min(b_1, b_2), \\ d &= \max(d_1, d_2), \\ u &= 1 - (b + d). \end{aligned} \tag{7.7}$$

This formulation reflects the fact that both unreliable features and mislabeled data can strongly bias parameter updates. The use of min and max provides a conservative aggregation: trust cannot exceed the weakest evidence, and distrust must reflect the strongest warning. It ensures that trust in both input features and labels is explicitly propagated into the parameter trust.

²The threshold ϵ is not a tuned hyperparameter but a sensitivity parameter used only in the NODE-TRUST function to distinguish weak from strong gradients. Its value can be chosen in different reasonable ways depending on the desired sensitivity.

Scale-Tied Threshold. The threshold ϵ used by NODETRUST separates weak from strong gradients, and its natural scale is that of the gradients themselves, which varies across tasks and across layers. Rather than treating ϵ as an absolute constant, we therefore also define a *scale-tied* variant: for each layer, $\epsilon_{\text{layer}} = c \cdot \text{median}(|\delta|)$, where the median gradient magnitude is estimated over the first revision batches of that layer and then frozen, and c is a single dimensionless constant ($c = 0.5$ unless stated otherwise). This makes the stability criterion relative to the task's own gradient scale: the same rule instantiates thresholds differing by nearly two orders of magnitude across the tasks studied here without any per-task tuning. Its empirical validation is reported in Section 7.10.

Computational Overhead during NN training. The proposed PaTAS-TP operates in parallel with the primary NN and mirrors its computational flow using subjective opinions. As a result, the overall execution time cost can be roughly expressed as:

$$\mathcal{C}_{\text{with PaTAS-TP}} \approx \mathcal{C}_{\text{comm}} + \max(\mathcal{C}_{\text{PaTAS-TP}}, \mathcal{C}_{\text{NN}}),$$

where $\mathcal{C}_{\text{comm}}$ denotes the communication overhead between the NN and the PaTAS-TP. Each PaTAS-TP operation is performed over subjective opinions represented by **multiple three stored** components (trust, distrust, uncertainty, ~~and a fixed base rate~~), leading to a higher computational cost compared to standard scalar operations. In practice, this results in $\mathcal{C}_{\text{PaTAS-TP}} \approx 4 \times \mathcal{C}_{\text{NN}}$. Consequently, the overall runtime is: the base rate is a shared constant), giving an analytical estimate of roughly $\mathcal{C}_{\text{PaTAS-TP}} \approx 3 \times \mathcal{C}_{\text{NN}}$ for the propagation arithmetic and a threefold memory overhead for the opinion parameters. Measured costs on an A100 GPU (784-512-10 architecture) are consistent with this estimate at inference time: trust propagation costs 0.43 ms per batch, amortizing from $14.8\times$ overhead at batch size 1 to $3.7\times$ at batch size 512. During training, wall-clock time is instead dominated by the synchronous gradient stream and revision when PaTAS-TP computation, yielding the approximation:-

$$\mathcal{C}_{\text{with PaTAS-TP}} \approx \mathcal{C}_{\text{comm}} + 4\mathcal{C}_{\text{NN}}.$$

~~Despite this overhead, the parallel design is attached in-process (roughly 7 versus 2200 optimization steps per second unattached), an engineering bottleneck of the socket-based prototype rather than of the method; since training requires no value from PaTAS-TP, the revision can equally run as an offline replay of the recorded gradient stream after training, which is how all cached experiments in this chapter were produced. The parallel design thus ensures that trust propagation does not interfere with the execution of the primary model, making the approach suitable for real-time monitoring scenarios where interpretability and reliability are critical. Moreover since the training does not need any value from , it can still continue while takes the time needed to complete its operations.-~~

7.7 Theoretical Properties of PaTAS-TP

To ensure that PaTAS-TP provides reliable and interpretable trust assessments, we first establish several fundamental theoretical properties describing its stability and consistency.

7.7.1 Convergence of PaTAS-TP

The convergence of the PaTAS-TP is governed by the stability of its inputs and the structure of its recursive Parameter-Trust Update process. During training, PaTAS-TP updates the internal trust opinions associated with network parameters using trust opinions on the inputs, labels, hyperparameters, and gradient information. This update mechanism is outlined in Alg. 6.

Theorem 7.1 (Convergence of PaTAS-TP Creation³). *Let a NN be trained until convergence, and let its associated PaTAS-TP operate with:*

- a stable input trust assessment T_x ,
- a stable label trust assessment T_y ,

Let $T_\theta^{(n)}$ denote the trust opinion at iteration n for parameter θ . If the revision of the trust in the weights is performed as specified in Alg. 6, then the PaTAS-TP parameter will converge. Specifically, if the revision operator $\ominus$ is a fusion operator, the sequence converges to the update opinion q (or remains at ω_0 if q is vacuous, as established in Theorem A.1); if $\ominus$ is the binomial multiplication operator $\odot$, the sequence converges componentwise, with disbelief accumulating toward 1 when $d_q > 0$ and belief decaying toward 0 when the projected probability $p_q < 1$, as established in Theorem A.2.

7.7.2 Symmetry and Invariance Properties of PaTAS-TP

Theorem 7.2 (PaTAS-TP Inference on Vacuous Input Yields Vacuous Output). *Let $\omega^\emptyset = (0, 0, 1, a)$ denote a vacuous binomial opinion over any variable, with arbitrary base rate $a \in [0, 1]$. Then the following two properties hold:*

1. Vacuous discounting: The discounting of a vacuous opinion is vacuous.
For any trust value binomial opinion ω_B^A , the discounted opinion $\omega_X^{[A;B]} = \omega_B^A \otimes (0, 0, 1, a) = (0, 0, 1, a)$
2. Vacuous Feedforward: PaTAS-TP Feedforward on a Vacuous Input is Vacuous.
Let $T_x = (0, 0, 1, a)$ be the trust assessment of an input to PaTAS-TP. Then for any parameter trust configuration T_θ , the output trust assessment satisfies: $T_y = \text{IPTA}(T_x) = (0, 0, 1, a)$.

Definition 7.4 (Symmetric Binomial Opinions). Let $\omega = (b, d, u, a)$ be a binomial opinion. The opinion $\bar{\omega} = (d, b, u, a)$ is called the *symmetric* of ω .

Theorem 7.3 (Symmetric Inference under PaTAS-TP). *Let T_θ be a PaTAS-TP feedforward function, and let $x = (b, d, u, a)$ be any binomial opinion with symmetric counterpart $\bar{x} = (d, b, u, a)$. Then the outputs $y = T_\theta(x)$ and $\bar{y} = T_\theta(\bar{x})$ are also symmetric, i.e., $y = (b', d', u', a)$, $\bar{y} = (d', b', u', a)$.*

In particular, the uncertainty is preserved: $u_y = u_{\bar{y}}$, and the belief in one output equals the disbelief in the other: $b_y = d_{\bar{y}}$, $d_y = b_{\bar{y}}$.

Remark 7.1. For a fully trusted input $x = (1, 0, 0)$, the discounting operator yields $\omega_\theta \otimes x = (P, 0, 1 - P)$, which has zero disbelief. Since fusion preserves zero disbelief across inputs, the output satisfies $d_y = 0$. By the same argument applied to $\bar{x} = (0, 1, 0)$, which produces zero belief after discounting, the output satisfies $b_{\bar{y}} = 0$.

³All proofs of the theorems in this section are provided in the appendices (Chapter A).

In other words, a fully trusted input $x = (1, 0, 0)$ yields an output with $d_y = 0$, and a fully distrusted input $\bar{x} = (0, 1, 0)$ yields an output with $b_{\bar{y}} = 0$.

This property illustrates a directional preservation of evidence: a fully trusted input cannot produce a non-zero disbelief mass at the output, regardless of the trustworthiness of the parameters. Symmetrically, a fully distrusted input cannot produce a non-zero belief mass at the output. Non-zero disbelief in the output can arise only from disbelief already present in the input; it is never introduced by the PaTAS-TP.

These properties serve as fundamental consistency checks, ensuring predictable behavior under neutral, vacuous, or balanced evidence.

7.8 Per-Sample Trust at Inference: Input Conformity and Composed Scores

The propagation machinery of the preceding sections consumes an input-trust opinion T_x but does not prescribe where it comes from. In the controlled experiments of Section 7.9, T_x is supplied externally as a scenario assumption (fully trusted, vacuous, or distrusted). For deployment, however, an externally supplied opinion is rarely available per input, and using a constant fully trusted opinion would make the propagated score carry no per-sample input information at all. This section instantiates the estimation-from-data route: input opinions are derived from each sample's conformity to the training-data statistics, then composed with the path trust into a single per-sample score.

7.8.1 Conformity-Based Input Opinions

Let the clean training features be $x^{(1)}, \dots, x^{(n)} \in \mathbb{R}^m$, with marginal mean μ_j and standard deviation σ_j per feature j , and let S be a robust span of the feature matrix (the difference between its 99.5th and 0.5th percentiles). The *effective deviation floors* each σ_j at a fraction of the span, $\sigma_j^{\text{eff}} = \max(\sigma_j, 0.02 S)$, so that near-constant features cannot produce unbounded standardized deviations from minimal perturbations. For an input x , the standardized deviation is $z_j = |x_j - \mu_j| / \sigma_j^{\text{eff}}$ and the conformity is

$$g_j = \exp\left(-\frac{1}{2} \max(z_j - 1, 0)^2\right) \in (0, 1],$$

so deviations within one effective standard deviation are fully conforming and only the excess erodes conformity. Each feature further receives a *reliability weight* $w_j = \min((\sigma_{\text{ref}} / \sigma_j^{\text{eff}})^2, 64)$ with σ_{ref} the median effective deviation: a feature with a tight training distribution is a reliable witness whose violation is unambiguous, while the cap bounds the influence any single feature can exert. Feature j contributes evidence $r_j = N w_j g_j$ and $s_j = N w_j (1 - g_j)$ with base evidence mass $N = 50$, mapped to a binomial opinion by the same Baseline-Prior Quantification used for parameter trust (prior weight $W = 2$, Chapter 6). The pooled per-sample input trust P_{input} is the projected probability of the opinion obtained from the summed evidence $(\sum_j r_j, \sum_j s_j)$, corresponding to cumulative fusion of independent evidence sources. All constants are fixed once and shared by every dataset in the evaluation.

Applicability Diagnostics. The construction is a per-feature marginal model, and its informativeness can be predicted *before any model is trained* from the weight statistics

it induces on the clean training features. Registered domains, where semantic content sits at fixed positions (centered digits, garment silhouettes), concentrate weight mass in tight, reliable witness features; unregistered natural imagery has no such witnesses, and the diagnostics predict that marginal conformity will be uninformative there. This prediction is borne out empirically in Experiment 4 (Table 7.7): the datasets with concentrated witnesses are exactly those where conformity detects corruption, and the boundary case flagged by the diagnostics is exactly where it fails. The diagnostics therefore serve as an a-priori deployment gate for the input-side signal.

7.8.2 Composed Per-Sample Score

At inference, three trust quantities are available for a sample x : the input conformity trust $P_{\text{input}}(x)$; the *path* trust $P_{\text{model}}(x)$, the projected probability of the predicted class's opinion obtained by feedforwarding a fully trusted input through the IPTA instantiated from x 's activation path, which isolates the reliability of the parameters actually used; and the model's own confidence $\hat{p}(x)$, its maximum softmax probability. The composed trust factor is the serial discount

$$P(x) = P_{\text{model}}(x) \cdot P_{\text{input}}(x),$$

the SL projected probability of trusting the prediction through both the path and the input source. For selective prediction the trust factor is combined with confidence through class-prior anchoring,

$$s(x) = P(x) \hat{p}(x) + (1 - P(x)) / K,$$

with K the number of classes: when trust is full the score is the model's confidence, and as trust vanishes the score falls to the uninformed prior $1/K$ rather than to an arbitrary zero. An alternative, fully propagated variant pushes the per-feature opinions through the IPTA feedforward instead of composing at the output; the averaging fusion inside the opinion product compresses the per-sample input signal, so serial composition is used as the default and the propagated variant is kept as an ablation.

Deployment Semantics. The composed score is a trust-weighted confidence, not a replacement for confidence. Confidence ranks correctness *within* the training distribution; trust flags inputs and parameters that *leave* it. The operational reading is therefore two-stage: gate on the trust factor (and on the applicability diagnostics), then rank by confidence among trusted inputs. The evaluation below makes this division of labor explicit and quantifies each factor's contribution separately.

7.9 Evaluation and Results

The goal of our evaluation is to validate the PaTAS-TP both theoretically and empirically. Specifically, we aim to demonstrate that PaTAS-TP produces interpretable trust estimates that (i) converge during training under specific conditions, (ii) respect symmetry and invariance properties, and (iii) are able to provide interpretable assessments under realistic conditions such as noisy features, corrupted labels, or adversarial perturbations, and (iv) provide per-sample and model-level signals whose

value against established uncertainty-quantification and detection baselines can be attributed to identifiable mechanisms.

Our evaluation approach combines controlled synthetic degradations with real-world datasets. We vary the trustworthiness of inputs ranging from fully trusted, fully uncertain, to fully distrusted, and observe how the created PaTAS-TP propagates these trust assessments through the network. For each scenario, we track three complementary metrics: trust mass (belief), uncertainty mass, and distrust mass (disbelief) of the trust opinion output by PaTAS-TP. We also compare these values against standard model accuracy (after training) to understand how input trust affects output reliability. We conduct ~~three~~ four experiments of increasing complexity ~~based on three different datasets:~~

1. *Breast Cancer Classification*, to assess behavior on a small, tabular medical dataset.
2. *MNIST Digit Classification*, to evaluate PaTAS-TP across multiple NN architectures under uncertainty.
3. *Poisoned MNIST*, to evaluate PaTAS-TP and IPTA in the presence of adversarial corruption and data poisoning.
4. *Comparative Trust Evaluation*, to evaluate the composed per-sample score of Section 7.8 against established uncertainty and detection baselines on Out of Distribution (OOD) and corruption detection, on fully connected and convolutional architectures, together with a test-data-free training-quality audit.

7.9.1 Experimental Setup

Experiment 1 - Breast Cancer Classification

We use the Breast Cancer Wisconsin (Diagnostic) Dataset [13], containing 569 samples with 30 numeric features (e.g., radius, area, symmetry) derived from breast mass fine needle aspirates. The NN classifies the tumors into benign or malignant categories. The NN architecture consists of 30 input neurons, 16 hidden neurons, and 2 output neurons, with ReLU activation in the hidden layer and Softmax in the output. The model is trained for 15 epochs with a batch size of 64 and a learning rate of 0.2, achieving 98% accuracy when the data are not modified.

For the evaluation, we degrade the training data in controlled ways and assign corresponding Subjective Logic trust assessments. We consider three extreme trust profiles (fully trusted (1,0,0), fully distrusted (0,1,0), and fully uncertain (0,0,1)) for both input features and label. These are combined (for inputs and label assessment) to form nine combinations. These dogmatic and vacuous opinions serve as canonical boundary cases in Subjective Logic: they express maximal trust, maximal distrust, and maximal uncertainty, respectively. We additionally include two intermediate scenarios, introduced below, to illustrate how PaTAS-TP behaves under partial trust and partial distrust.

In practice, feature and label trust degradation may arise from poor-quality imaging, human annotation errors, or flaws in the preprocessing pipeline. In this experiment and the following ones, we simulate degradations using controlled perturbation functions. For fully uncertain feature opinion, we introduce additive uniform noise

$$x' = x + \mu, \quad \mu \sim U(-\eta, \eta),$$

where $\eta = 0.3 \times \max(\text{features})$. Each feature is perturbed with probability 0.3. After noise addition, if x' lies outside the valid range $[\min(\text{features}), \max(\text{features})]$, it is clipped to the corresponding boundary:

$$x' = \begin{cases} \min(\text{features}), & x' < \min(\text{features}), \\ \max(\text{features}), & x' > \max(\text{features}), \\ x', & \text{otherwise.} \end{cases}$$

To model distrust, we generate corrupted inputs by sampling from a uniform distribution over the feature space:

$$x' \sim U(\min(\text{features}), \max(\text{features})),$$

Label degradation is modeled analogously: uncertainty is introduced via random label noise, while full distrust is represented by a complete replacement of the labels.

Finally, to complement the boundary cases, we evaluate two intermediate trust scenarios:

- i. the same scenario for a fully uncertain input features and fully uncertain labels, but where the assessment is set to $(0.25, 0.25, 0.5)$, reflecting partial distrust and trust, instead of a fully uncertain opinion $(0, 0, 1)$ and,
- ii. a mild degradation, where features and labels are perturbed with probability 0.15 instead of 0.3 and the trust assessment is set to $(0.25, 0, 0.75)$.

Experiment 2 - MNIST

In this experiment, we evaluate the PaTAS-TP framework on the MNIST dataset [14], which consists of 60,000 training images and 10,000 test images, each representing a digit from 0 to 9. Each image has 784 features (28x28 pixels). The NN classifies these images into one of the 10 digit classes. We test five NN architectures:

- Architecture 1: 16 hidden neurons (784-16-10).
- Architecture 2: 32 hidden neurons (784-32-10).
- Architecture 3: 64 hidden neurons (784-64-10).
- Architecture 4: 128 hidden neurons (784-128-10).
- Architecture 5: 2 hidden layers with 16 neurons each (784-16-16-10).

Although these architectures are not state-of-the-art for MNIST, they are sufficient to evaluate the behavior of PaTAS-TP across different model sizes and demonstrate how trust propagates through the network. Each model uses the ReLU activation function for the hidden layer and Softmax for the output. Models are trained for 20 epochs with a batch size of 128 and a learning rate of 0.05, achieving test accuracies of up to 98%. We evaluate using fully uncertain training data trust assessment functions for both input features and labels, corresponding to the case where we have no knowledge about the dataset. Additionally, to contrast with the uncertain case, we perform a supplementary evaluation using a fully trusted assessment on Architecture 1 and 5.

Experiment 3 - Poisoned MNIST

In this experiment, we evaluate the PaTAS-TP framework on a poisoned version of the MNIST dataset, where one third of the training images are corrupted: labels of digits 6 and 9 are flipped, and at the same time a visible patch of fixed size is added at the top-left corner of the corresponding images. This combination follows common practices in data poisoning and backdoor attack scenarios [15]. The remaining two thirds of the data remain clean. This setup allows us to examine how PaTAS-TP responds to the simultaneous presence of corrupted labels and adversarial triggers that can undermine both model performance and trust. We use Architecture 4 from the previous experiment, consisting of an input layer with 784 neurons, a hidden layer with 128 neurons, and an output layer with 10 neurons (784-128-10).

For the poisoned dataset, trust is assigned as follows:

- Pixels corresponding to the patch are considered distrusted, while all other pixels are trusted.
- Labels for patched images of digits 6 and 9 are distrusted, while others are trusted.

This setup helps the PaTAS-TP framework focus on potentially corrupted areas, while trusting the unaffected parts of the data.

Experiment 4 - Comparative Trust Evaluation

We compare the composed per-sample PaTAS-TP score against maximum softmax probability, MC Dropout [16] ($T=30$, dropout 0.2), and Evidential Deep Learning (EDL) [17] (ranking metrics score its negated Dirichlet total uncertainty $1 - u$ with $u = K/S$; calibration metrics its maximum expected probability), each trained with the modifications its method requires under the shared architecture, optimizer, and epoch budget. To attribute mechanisms rather than merely outperform output-only scores, we further include detector baselines in both representation spaces: the class-conditional Mahalanobis distance [18] and the deep nearest-neighbor distance [19] on penultimate features, the energy score [20] on logits, their raw-pixel counterparts (Mahalanobis with shrinkage covariance and k -nearest-neighbor distance fit directly on training pixels), and the conformity score P_{input} alone, i.e., the input opinions of Section 7.8.1 with all propagation removed.

Primary datasets are MNIST and Fashion-MNIST [21] (784-512-10, 20 epochs); GTSRB [22] (1024-128-43, train-standardized) serves as an applicability-boundary case flagged in advance by the diagnostics of Section 7.8.1. The battery is additionally run on convolutional networks with the conv IPTA of Section 7.5.1: residual CNNs on MNIST and Fashion-MNIST (99.3% and 88.7% test accuracy) and a deeper residual CNN on CIFAR-10 ($84.5\% \pm 0.8$); the networks use no normalization layers, so trust propagation covers every parameter. Every method is evaluated identically on clean test data, under Bernoulli-uniform feature corruption, on mixed batches pooling clean and corrupted inputs 1:1, and as an OOD detector (Fashion-MNIST for MNIST and vice versa, identical preprocessing). Beyond uniform noise, two harder paired corruptions probe the conformity model at its declared blind spots: a random circular translation of up to two pixels (marginal-preserving) and a low-amplitude trigger alpha-blended into the image center (an adaptive adversary staying within the feature range). Post-hoc isotonic calibration is fit per method on a held-out clean split; reported detection metrics are the area under the ROC curve (AUROC) for separating the relevant classes of inputs, as mean over three evaluation seeds (seed dispersion

at most ± 0.005 unless stated). The training-quality audit trains the MNIST network under increasing label-flip rates and reads the model-side trust with no test data; it is complemented by a dead-unit confound check and by single-channel poisoning controls (label flip only, trigger patch only) for Experiment 3's combined corruption.

Implementation Details for All Experiments During inference, multiplication operations in the feedforward phase are implemented using SL trust discounting, while addition operations employ a generalized SL averaging fusion operator to support summations over multiple inputs (see Definition 7.3). Trust revision uses the same averaging fusion. The PaTAS-TP framework and NN were implemented in Python 3.13 using NumPy. Two main modules were developed: PrimaryNN, which handles NN structure, training, and inference, and PaTAS-TP, which implements propagation functions defined in the operational flow (Fig. 7.2). These modules interact to dynamically evaluate and update trust during feedforward and backpropagation. The comparative battery of Experiment 4 additionally implements the baseline methods in PyTorch within the same repository, sharing data loading, preprocessing, and evaluation code across all methods. The full implementation, including all modules, dependencies, and experiment scripts, is available on github, with detailed instructions for reproducing all experiments.

7.9.2 Results and Analysis

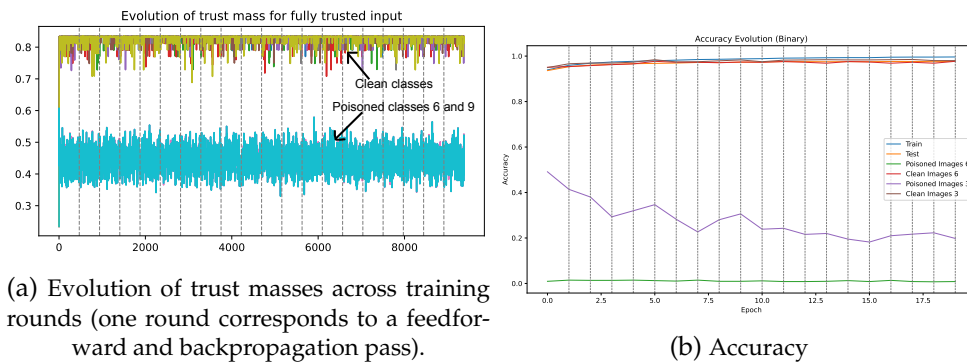


FIGURE 7.6: Evolution of trust mass for fully trusted input in the MNIST poisoned Experiment with patch size 4×4 pixels

The evaluation of the Parallel Trust Assessment System (PaTAS) is conducted during the training phase of the NN. After each iteration of training (i.e., a complete feedforward and backpropagation cycle), we assume that an inference is performed, and we assess the trustworthiness of the corresponding output. This assessment is carried out under three input trust profiles:

- *Fully Trusted Input*: input is considered fully trusted.
- *Fully Uncertain Input*: input is considered fully uncertain.
- *Fully Distrusted Input*: input is fully unreliable.

We focus on these three profiles because they represent the extreme and most informative boundary cases of input trust.

For each of these three input types, we track and plot the evolution of three key metrics over the course of training:

- *Trust Mass*: the level of confidence in the output.
- *Uncertainty Mass*: the uncertainty of the assessment.
- *Distrust Mass*: the level of disbelief in the output.

As a result, for each evaluation, 9 distinct plots are generated, corresponding to the 3 input types (fully trusted, fully uncertain, and fully distrusted) and each of the three key metrics (trust, uncertainty, and distrust)⁴.

All the results are depicted in Figures in the appendices and summarized in Tables 7.1, 7.3 and 7.4 and Fig. 7.7. They confirm several theoretical properties of PaTAS-TP proven in Section 7.7:

- *Convergence of Trust Assessment*: when accuracy converges, trust assessment converges as expected.
- *Inference on fully uncertain input yields fully uncertain output*: a fully uncertain input always produces a fully uncertain output.
- *Symmetric Inference*: when input trust assessments are symmetric, the inference results remain symmetric.

For detailed results, we primarily focus on the three plots for each experiment) corresponding to feedforward of fully trusted input. Processing uncertain inputs naturally yields uncertain outputs, while symmetry implies that trusted and distrusted cases mirror each other. We also confirm that when processing fully trusted inputs, the distrust mass remains close to zero. Since trust, uncertainty, and distrust sum to 1, analyzing only the trust mass is sufficient in such cases. Therefore we keep only the trust mass after the training in Tables 7.1 to 7.4.

Experiment 1 – Breast Cancer Classification

X Trust	Y Trust	$\epsilon = 1$	$\epsilon = 0.1$	$\epsilon = 0.01$	$\epsilon = 0.001$	Train (%)	Test (%)
trust	trust	0.8787	0.8787	0.6658	0.1681	98.68	96.49
trust	vacuous	0.3242	0.2927	0.1290	0.0773	65.71	74.56
trust	distrust	0.00	0.00	0.00	0.00	99.12	2.63
vacuous	trust	0.3392	0.3392	0.2682	0.0931	96.70	95.61
vacuous	vacuous	0.3051	0.2971	0.1820	0.1034	68.79	88.60
vacuous	distrust	0.00	0.00	0.00	0.00	96.92	4.39
distrust	trust	0.00	0.00	0.00	0.00	64.62	43.86
distrust	vacuous	0.00	0.00	0.00	0.00	56.92	33.33
distrust	distrust	0.00	0.00	0.00	0.00	65.49	40.35
<i>Extra scenarios</i>							
(0.25, 0.25, 0.5)	(0.25, 0.25, 0.5)	0.1172	0.1140	0.0633	0.0272	68.79	88.60
(0.25, 0.0, 0.75)	(0.25, 0.0, 0.75)	0.3810	0.3809	0.2211	0.0871	80.22	95.61

TABLE 7.1: Summary of final trust mass across four values of ϵ and corresponding train/test accuracies for the Breast Cancer Classification Experiment.

The results for this experiment are summarized in Table 7.1. When both features and labels are clean, the model steadily improves and achieves high accuracy (98.68% training, 97.69% test), and PaTAS-TP produces the highest trust mass across

⁴The base rate is set to 0.5 and remains constant across all experiments.

all conditions (0.88 for $\epsilon = 0.1$). Clean features with corrupted labels cause test accuracy to collapse to 2.63%, even while training accuracy remains artificially high at 99.12%; the model learns the wrong mapping but does so confidently. Correspondingly, trust mass falls to zero regardless of ϵ , correctly signaling the unreliability of the learned parameters. With vacuous labels, performance degrades moderately (65.71% train, 74.56% test) and trust mass remains low (0.29). Corrupted features eliminate the ability to learn in any combination: trust mass is zero in all distrust- X rows and test accuracy never exceeds 43.86%.

The mild-degradation scenario (0.25, 0, 0.75) for features and labels is the only intermediate case and behaves as expected: with only 15% of features perturbed and no distrust component, trust mass reaches 0.38 for $\epsilon = 0.1$, outperforming every row that uses a fully vacuous opinion for either features or labels. This suggests that an analyst who can bound the degradation and express it as a partial-trust opinion, rather than defaulting to full uncertainty, obtains a more optimistic trust assessment.

We evaluated the system for four values of ϵ (0.001, 0.01, 0.1, 1). We recall that ϵ is the threshold used by the NODETRUST function in Alg. 6.

- For $\epsilon = 0.01$: the system is more sensitive to gradient magnitude. Fully trusted X with trusted Y yields a trust mass of 0.67, while uncertain Y reduces it to 0.13. When X is vacuous, trust stabilizes around 0.27 for trusted Y and 0.18 for uncertain Y .
- For $\epsilon = 0.1$: the larger threshold makes the system less reactive to small gradients. Fully trusted X with trusted Y reaches 0.88; with uncertain Y , trust falls to 0.29. Vacuous X with trusted Y yields 0.34, and with uncertain Y yields 0.30. In all cases, distrust in either X or Y drives trust mass to zero.

The two additional values clarify the boundary behavior of ϵ . At $\epsilon = 0.001$, the threshold falls well below the typical gradient magnitude during training, causing NODETRUST to classify nearly all gradients as large and therefore as negative evidence; trust mass collapses across all non-distrust configurations, losing the ability to differentiate between trust profiles. At $\epsilon = 1$, the threshold is so permissive that almost all gradients are treated as small, producing results nearly identical to $\epsilon = 0.1$ and offering no additional discriminative power. Among the four values evaluated, $\epsilon = 0.1$ emerges as the most balanced choice, maintaining sensitivity to meaningful gradient activity without collapsing trust mass or losing discriminative power. How ϵ should be set in practice is discussed in Section 7.10.

A notable observation is that the *trust/vacuous* configuration yields a lower trust mass than *vacuous/vacuous* across all values of ϵ except $\epsilon = 1$. This ordering is consistent with the test accuracies reported in Table 7.1, where *trust/vacuous* (74.56%) underperforms *vacuous/vacuous* (88.60%). The underlying mechanism is that trusted features produce strong, informative gradients that frequently exceed ϵ , generating negative evidence in NODETRUST when labels are unreliable: the model confidently learns from clean features under vacuous supervision, effectively fitting an unreliable mapping. Noisy features, by contrast, produce diffuse gradients that remain below ϵ more often, partially canceling the effect of uncertain labels and yielding comparatively higher trust mass. This shows that smaller values of ϵ correctly capture this ordering, while $\epsilon = 1$ is too permissive to discriminate between these regimes and the effect vanishes.

Overall, there is a strong positive correlation between trust mass and test accuracy. Moreover, a smaller ϵ makes label trust more influential: the relative gap between trusted- Y and uncertain- Y rows, defined as $(T_{\text{trusted}} - T_{\text{uncertain}})/T_{\text{trusted}}$, increases as ϵ decreases, as shown in Table 7.2.

TABLE 7.2: Relative influence of label trust as a function of ϵ , for trusted and vacuous feature inputs.

X trust	$\epsilon = 0.1$		$\epsilon = 0.01$	
	$T_{Y=\text{trust}}$	$T_{Y=\text{vac}}$	$T_{Y=\text{trust}}$	$T_{Y=\text{vac}}$
trusted	0.88	0.29	0.67	0.13
vacuous	0.34	0.30	0.27	0.18
Relative gap (trusted): $(0.88 - 0.29)/0.88 = 67\%$ vs $(0.67 - 0.13)/0.67 = 81\%$				
Relative gap (vacuous): $(0.34 - 0.30)/0.34 = 12\%$ vs $(0.27 - 0.18)/0.27 = 33\%$				

Architecture	X Trust	Y Trust	Trust Mass	Train Acc (%)	Test Acc (%)
16	vacuous	vacuous	0.3087	65.19	73.96
32	vacuous	vacuous	0.3224	66.20	79.03
64	vacuous	vacuous	0.3306	67.22	89.98
128	vacuous	vacuous	0.3359	67.78	92.05
16-16	vacuous	vacuous	0.1628	65.68	69.39
16	trust	trust	0.8620	96.37	95.13
16-16	trust	trust	0.7077	96.83	95.84

TABLE 7.3: Summary of final trust mass for the MNIST Classification Experiment.

Key findings – Experiment 1

F1.1 Label trust matters more than feature trust. Setting X as vacuous while keeping Y trusted yields a higher trust mass (0.34) than the reverse (0.29 for trusted X, uncertain Y).

F1.2 Distrust is more damaging than uncertainty. Replacing a fully uncertain opinion $(0, 0, 1)$ for both X and Y with a mixed assessment $(0.25, 0.25, 0.5)$ reduces trust mass from 0.30 to 0.11, while accuracy remains unchanged (88.60%), confirming the drop is driven by the distrust component alone.

F1.3 Finer ϵ amplifies label-trust sensitivity. As shown in Table 7.2, the relative gap between trusted-Y and uncertain-Y is larger for $\epsilon = 0.01$ than for $\epsilon = 0.1$, in both the trusted and vacuous feature settings.

Experiment 2 – MNIST Dataset

In this experiment, PaTAS-TP produces ten output trust opinions, one for each class. However, they are almost identical across classes, so we record only the aggregated trust mass in Table 7.3.

When features and labels are vacuous, accuracy improves as model size increases, from 73.96% (architecture 16) to 92.05% (architecture 128). Test accuracy consistently exceeds training accuracy across all architectures, suggesting that training under vacuous trust makes the model underestimate its own learning progress. Trust mass follows the same upward trend with model size (0.3087, 0.3224, 0.3306, 0.3359), though the gains become progressively smaller, indicating diminishing returns from added capacity under uncertain data.

The two-layer architecture (16-16) is a notable exception. Despite having more parameter count than the single-layer 16-hidden neurons network, it yields a trust mass of only 0.1628 under vacuous inputs and 0.7077 under trusted inputs, both

consistently lower than the single-layer 16-neuron counterpart (0.3087 and 0.8620 respectively). This is not an artifact but a meaningful property of the framework: each additional layer introduces an extra round of trust discounting, which attenuates the trust signal propagating through the network. A deeper architecture therefore inherits less trust from its inputs, even when those inputs are reliable. This behavior is consistent with PaTAS-TP design, since a deeper inference path involves more parameters, each contributing uncertainty, and the framework correctly reflects that the output trust depends on the full chain of parameter reliability. A secondary factor is convergence speed: deeper networks require more epochs to stabilize their gradients, so the parameter trust opinions accumulate evidence more slowly and remain closer to their vacuous initialization at the end of training.

The trusted evaluation provides the clearest contrast. Architecture 16 with trusted features and labels reaches a trust mass of 0.8620, far exceeding any vacuous-data configuration regardless of architecture size. Even the deeper 16-16 architecture achieves 0.7077 under trusted inputs, outperforming the largest single-layer architecture (128, trust mass 0.3359) trained under vacuous data by a wide margin. This confirms that data quality has a far greater impact on the resulting trust assessment than model capacity.

Key findings – Experiment 2

F2.1 Trust increases with model width, with diminishing returns. Under vacuous data, trust mass grows from 0.3087 (16 neurons) to 0.3359 (128 neurons), but the incremental gain shrinks at each step, suggesting that additional capacity provides less benefit once the dominant bottleneck is data quality rather than model expressiveness.

F2.2 Depth penalises trust due to discounting accumulation. The 16-16 architecture consistently yields lower trust than the single-layer 16-neuron network under both vacuous (0.1628 vs. 0.3087) and trusted (0.7077 vs. 0.8620) inputs. Each additional layer introduces an extra discounting step, so deeper networks attenuate the trust signal regardless of their predictive accuracy.

F2.3 Data quality dominates model capacity for trust. Training the smallest architecture on trusted data (trust mass 0.8620) far exceeds training the largest architecture on vacuous data (trust mass 0.3359). Investing in data quality is therefore more beneficial for trust than scaling the model.

Experiment 3 – Poisoned MNIST Dataset

Patch size	Trust for 3	Trust for 6	Train (%)	Test (%)	Clean 3 (%)	Clean 6 (%)	3 with patch (%)	6 with patch (%)
(1 × 1)	0.836	0.440	99.62	97.66	97.72	97.49	83.76	0.63
(4 × 4)	0.833	0.439	99.62	97.66	98.02	97.70	19.80	0.84
(10 × 10)	0.815	0.433	99.60	97.61	97.62	97.49	16.44	0.63
(27 × 27)	0.651	0.386	96.35	97.73	98.02	97.81	0.00	0.00

TABLE 7.4: Summary of results for the Poisoned MNIST Classification Experiment.

As in the previous experiment, PaTAS-TP produces ten output trust opinions. In Table 7.4 we report the trust mass for label 3 (a clean class) and label 6 (a poisoned class), alongside overall train/test accuracy and per-class accuracy on both clean and patched test samples.

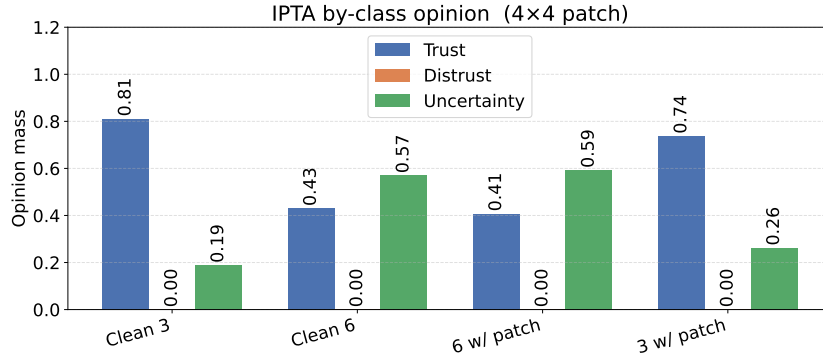


FIGURE 7.7: IPTA results for NN trained on poisoned datasets with patch size 4×4 . Results are obtained by feedforwarding a fully trusted opinion, except the patch-distrusted, where patch pixels are explicitly assigned distrust.

TABLE 7.5: Relative trust gap between clean label 3 and poisoned label 6 as a function of patch size.

Patch size	T_3	T_6	$(T_3 - T_6)/T_3$ (%)
1×1	0.836	0.440	47.4
4×4	0.833	0.439	47.3
10×10	0.815	0.433	46.9
27×27	0.651	0.386	40.7

Across all patch sizes, the trust mass for label 3 consistently exceeds that for label 6 (e.g. 0.836 vs. 0.440 for the 1×1 patch), confirming that PaTAS-TP reliably distinguishes clean from poisoned labels even when overall accuracy remains high (97.66%). As the patch grows, both trust values decrease, but not symmetrically: as shown in Table 7.5, the relative gap $(T_3 - T_6)/T_3$ decreases with patch size, from 47.4% at 1×1 down to 40.7% at 27×27 , indicating that larger patches erode the trust advantage of the clean class more than they further penalize the already-distrusted poisoned class. As a result, the gap between the two patches decreases with patch size. This asymmetry is interpretable: as the patch occupies a larger fraction of the image, the underlying model increasingly associates the patch pattern directly with the target poisoned class, bypassing the remaining image features. The parameters involved in clean-class prediction are therefore affected more by the growing patch than those already conditioned on the poisoned signal, which partially absorbs the patch regardless of its size. The clear separation between the two classes is nonetheless maintained across all patch sizes.

Regarding classification accuracy, patched digit 6 samples are almost entirely misclassified regardless of patch size ($\leq 0.84\%$), confirming the backdoor attack is effective. Patched digit 3 accuracy degrades more gradually, dropping sharply from 83.76% at 1×1 to roughly 16 to 20% for larger patches, reflecting the spatial overlap between the patch and discriminative features for digit 3. The 27×27 case is an edge case rather than a realistic poisoning scenario: a patch of this size dominates the entire image at inference time, so the model associates the input almost entirely with the poisoned trigger rather than the digit. One of the poisoned classes therefore reaches near-100% accuracy while the other collapses to 0%, producing the 0% accuracy observed for both patched digits 3 and 6. This extreme case is included only to illustrate the boundary behavior of the framework.

To evaluate the IPTA, Fig. 7.7 reports accuracy and the full trust opinion (t, d, u) for clean and patched samples, obtained by feedforwarding a fully trusted opinion through the IPTA instantiated from the respective inference path in the 4×4 patch model. As established in Theorem 7.2 (Remark 7.1), feedforwarding a fully trusted input $(1, 0, 0)$ through any PaTAS-TP configuration yields an output with zero distrust mass, since the discounting operator maps a fully trusted input to an opinion with $d = 0$, and fusion preserves this zero mass. This is confirmed across all rows using a trusted T_x : distrust remains exactly 0 in every case.

Clean digit 3 achieves the strongest trust profile (0.811 trust, 0.189 uncertainty), consistent with high accuracy (98.02%). Clean digit 6 already exhibits markedly higher uncertainty (0.570) despite comparable accuracy (97.70%), reflecting that the parameters on the digit-6 inference path were exposed to poisoned training samples and are therefore less trusted.

For patched samples, the degradation is clear. Patched digit 6 accuracy collapses to 0.84% and trust drops to 0.408, with uncertainty dominating at 0.592, providing a warning signal unavailable from accuracy alone. For patched digit 3, accuracy falls to 19.80% yet trust remains relatively higher at 0.738, reflecting that the digit-3 inference path passes through less poisoned parameters. [A ten-training replication of this setup under the released implementation, with only the evaluation seed varied, confirms the gap is robust: mean IPTA trust \$0.884 \pm 0.001\$ for the clean class against \$0.503 \pm 0.035\$ for the poisoned class. Two assumptions of this experiment, the oracle knowledge of the patch location in the trust assignment and the joint application of patch and label flip, are removed in Experiment 4, which detects the same poisoning with estimated input opinions and separates the two corruption channels.](#)

Key findings – Experiment 3

F3.1 Trust separates clean from poisoned inference paths. Across all patch sizes, the trust mass for label 3 (clean) consistently exceeds that for label 6 (poisoned), even when overall test accuracy remains above 97%. PaTAS-TP therefore reveals reliability differences that accuracy alone cannot expose.

F3.2 Larger patches narrow the trust gap by affecting clean paths more than poisoned ones. As patch size grows, the trust mass for the clean label decreases more steeply than for the poisoned label. The model increasingly relies on the patch pattern rather than image features, which degrades the parameters along clean-class inference paths more than those already conditioned on the poisoned signal.

F3.3 Zero distrust under trusted input is theoretically guaranteed. When the input trust is fully trusted $(1, 0, 0)$, distrust in the output is exactly zero, as established in Theorem 7.2. This is confirmed empirically across all IPTA rows using trusted T_x .

These results demonstrate that PaTAS-TP not only correlates with accuracy but provides complementary information: it identifies which inference paths are structurally less reliable, responds to the size of adversarial corruption, and distinguishes clean from poisoned classes even when overall accuracy remains high.

Experiment 4 – Comparative Trust Evaluation

In-distribution behavior. [On clean test data the composed score matches the maximum-softmax filter on correct-versus-incorrect discrimination \(AUROC \$0.979 \pm 0.003\$ for both on](#)

MNIST), with equal calibration error after identical post-hoc calibration: when nothing violates the training distribution, trust-weighting costs nothing. Under corruption the two signals separate *by design*: accuracy barely drops on the robust MLP, so the softmax retains a higher correct-versus-incorrect AUROC on the mixed batch (0.974 vs. 0.895) precisely because it ignores the distribution violation that the trust factor flags. Confidence ranks correctness within the training distribution; trust exposes inputs that leave it. The detection tasks below measure the latter.

Method	OOD M→F AUROC	Det. MNIST AUROC	Det. F-MNIST AUROC	OOD F→M AUROC	Shift AUROC	Blend AUROC
PaTAS (composed)	0.975	0.972	0.835	0.781	0.781	0.564
Conformity alone	1.000	1.000	0.994	0.481	0.678	0.500
Mahalanobis (pixel)	0.997	0.997	0.944	0.942	0.716	0.482
kNN (pixel)	0.965	0.603	0.707	0.967	0.749	0.500
Mahalanobis (feature)	0.978	0.743	0.785	0.988	0.783	0.474
kNN (feature)	0.994	0.588	0.625	0.973	0.801	0.508
Energy score	0.850	0.636	0.603	0.873	0.707	0.541
Max softmax	0.883	0.631	0.558	0.805	0.740	0.566
MC Dropout	0.936	0.621	0.545	0.834	0.747	0.574
EDL	0.883	0.547	0.510	0.767	0.736	0.561

TABLE 7.6: Detection of distribution violations (mean over three evaluation seeds, dispersion at most ± 0.005): OOD detection in both directions between MNIST and Fashion-MNIST, clean-versus-corrupted separation on the mixed batches (uniform feature noise), and the two hard MNIST corruptions (marginal-preserving translation; low-amplitude blended center trigger). The blended trigger defeats every method (best entry 0.574), an honest limit of all ten scores against an adversary that stays inside the per-feature range.

Detection and attribution. Table 7.6 reports the violation-detection tasks. Three mechanisms are cleanly separable. First, *violation detection is carried by input-space signals*: the conformity opinions are perfect or near-perfect where inputs violate the training marginals (OOD M→F 1.000, corruption detection 1.000 and 0.994), the raw-pixel Mahalanobis baseline is close behind, and an ablation replacing the conformity opinions with the constant fully trusted opinion collapses the composed score to the output-only level (0.896 and 0.639 on the forward tasks, single-seed). Trust propagation is not the detector; it is the carrier that turns the detection signal into an opinion composable with model trust. Second, *propagation contributes exactly where the input marginals are silent*: under the marginal-preserving translation the conformity score drops to 0.678 as predicted by its construction, while the composed score reaches 0.781, above its own input factor, because translated inputs traverse atypical activation paths that the IPTA reads as less trusted. Third, *the directions of failure are principled*: in the subset direction (Fashion-MNIST model, MNIST digits as OOD), digits occupy a quiet subset of the clothing marginals rather than violating them, the conformity score collapses below chance (0.481), and the neighborhood- and covariance-based detectors own the task (0.988 feature Mahalanobis, 0.942 pixel Mahalanobis). The composition fails safely there: the confidence factor keeps the composed score at the output-baseline level (0.781 vs. 0.805 softmax) instead of inheriting the collapse. Since the subset-direction signal demonstrably exists in input space, mapping such a detector through the same quantification into an additional input opinion is a natural SL extension, left as future work.

On the applicability-boundary case, GTSRB, the marginal conformity model detects corruption only weakly (0.645; composed 0.545), exactly as the diagnostics of Section 7.8.1 predict (Table 7.7: no witness features with $w_i \geq 4$). The pixel-space Mahalanobis baseline still reaches 0.997 on the same task: the boundary belongs to the per-feature marginal model, not to input-space assessment as such, which sharpens the diagnostics into an actionable gate: when they flag a domain, a covariance-aware input model is required.

Dataset	Med. σ_{eff}	w_{max}	Top-10%	$w > 4$	Det.
MNIST	0.47	52.3	0.32	0.40	0.972
Fashion-MNIST	0.90	64.0	0.76	0.18	0.835
GTSRB	0.98	1.8	0.14	0.00	0.545
CIFAR-10 (gray)	0.23	1.1	0.11	0.00	—

TABLE 7.7: A-priori applicability diagnostics of the conformity model, computed on clean training features before any model is trained (median effective deviation, maximum reliability weight, weight share of the top decile, fraction of features with $w_i \geq 4$), against the composed score’s corruption detection later observed. Registered domains concentrate evidence in reliable witnesses; unregistered imagery has none, and the input-side signal is predicted, and confirmed, to be uninformative there.

Poisoning without an oracle. The conformity opinions remove the oracle assumption of Experiment 3. Fed unchanged into the poisoned model’s IPTA, they flag patched inputs with no knowledge of the trigger: the input trust alone separates clean from patched inputs at AUROC 1.000 for both classes, because the patch violates the near-constant border features the conformity model weights most, and the parameter-side gap between the clean and poisoned class persists under these estimated opinions (trust 0.780 vs. 0.499; oracle assignment 0.871 vs. 0.559). The remaining assumption, a clean feature source for the conformity fit, degrades gracefully: fitting on the poisoned training set itself still detects the trigger at 0.73 and 0.67 per class, while a naive robust fit (median and MAD) does not recover (0.65), since MNIST pixel marginals are zero-inflated; contamination-aware conformity fits are explicit future work.

Separating the corruption channels. Experiment 3 applies patch and label flip jointly. Single-channel controls retrained at the paper’s patch size separate their effects on the per-class output trust (Table 7.8): under the full backdoor both flipped classes carry an identical trust deficit (0.756 for digits 6 and 9 alike, 0.19 below the minimum of the eight clean classes, whose spread is below 0.001); the label-flip-only control reproduces nearly the full drop (0.748), the patch-only control still produces a clearly detectable deficit (0.788), and the unpoisoned model shows no false alarm (both classes at the clean level 0.994). The trust deficit is therefore pair-specific, symmetric across the flipped pair, driven primarily by the label channel, and present for a purely spurious trigger even when every label is correct.

Training-quality audit. The model-side trust provides a training-quality audit that requires no test data at all (Table 7.9). Under increasing train-time label-flip rates p , the feedforward trust under a fully trusted input drops by 31.7% relative to the clean-trained model at *any* non-zero rate, including $p = 0.1$ where test accuracy

Training condition	Clean-class mean	Clean-class min	Trust ₆	Trust ₉
Unpoisoned	0.994	0.994	0.994	0.994
Full backdoor (patch + flip)	0.947	0.946	0.756	0.756
Label flip only	0.947	0.947	0.748	0.747
Trigger patch only	0.948	0.948	0.788	0.788

TABLE 7.8: Per-class output trust (projected probability under a fully trusted input) for the 4×4 poisoned MNIST setup and its single-channel controls (784-128-10). Both flipped classes drop identically and far outside the clean-class band; the label channel carries most of the deficit, and the spurious patch alone remains detectable.

moves by only three points: a corruption tripwire read from the trained trust state alone. The flat level is designed saturation, not an artifact: at $p = 0.1$ essentially every training batch already contains corrupted labels, so every batch delivers the large-gradient corrections that NODETRUST counts as negative evidence, and the converged trust registers *that* corruption is present rather than how much. When the algorithm instead consumes rate-informed label opinions $T_y = (1-p, 0, p)$, the same machinery grades monotonically ($0.951 \rightarrow 0.805$): tripwire versus graded meter is a property of the label opinion’s informativeness, not of the calculus. The scale-tied threshold of Section 7.6 preserves the tripwire without any tuning: at $\epsilon_{\text{layer}} = 0.5 \cdot \text{median}(|\delta|)$ the clean model reads 0.625 against a flat 0.545 to 0.550 at every non-zero rate, a margin more than ten times the within-rate spread, while the same rule instantiates thresholds from $5 \cdot 10^{-5}$ (MNIST) to $3 \cdot 10^{-3}$ (breast cancer) across layers and tasks and preserves the trust ordering of the cancer grid. Data-centric baselines frame the comparison honestly in both directions: the fraction of training labels disagreeing with confident predictions recovers p almost exactly and mean training loss grades likewise, but both require retaining the training data at audit time, and mean confidence measures severity but needs clean held-out data.

A dead-unit confound check completes the audit’s validation. At every flip rate, no hidden unit is inactive on more than 99% of inputs (dead-unit fraction 0.0%), the mean trust of active units itself drops from 0.9999 to 0.626 at any non-zero rate, an activity-weighted variant of the audit that removes low-activity units’ contribution by construction reproduces the standard audit to within 0.001 at every rate, and the correlation between unit activation frequency and unit trust is essentially zero. The audit signal lives in the live paths of the network; it does not measure network degeneration.

p	Acc. (%)	Conf.	Trust	Trust _{inf}	Trust _{auto}	Disagree	Train loss
0	97.9	0.981	0.998	0.998	0.625	0.001	0.014
0.1	94.7	0.841	0.682	0.951	0.545	0.099	0.419
0.3	93.7	0.684	0.682	0.872	0.550	0.288	1.013
0.5	82.8	0.476	0.681	0.805	0.546	0.461	1.472

TABLE 7.9: Training-quality audit (MNIST, 784-512-10) under label-flip rate p . Trust is the feedforward trust under a fully trusted input (no test data needed) and acts as a corruption tripwire; Trust_{inf} uses rate-informed label opinions $(1-p, 0, p)$ and grades monotonically; Trust_{auto} is the tripwire under the scale-tied threshold ($c = 0.5$, no tuning); Disagree and train loss need the training data at audit time; confidence needs clean held-out data.

Convolutional architectures. The battery transfers to CNNs through the activity-weighted conv IPTA of Section 7.5.1; per-sample scoring never failed across all runs (Table 7.10). On the 99.3% MNIST CNN the composed score matches the softmax filter on clean correct-versus-incorrect ranking (0.986 for both) while dominating detection (corruption 0.988 vs. 0.662; OOD 0.991 vs. 0.967); on this architecture the composition pays no in-distribution price at all. EDL, weak on the MLPs, becomes the best in-distribution ranker on the CNN (clean AUROC 0.996) yet stays blind to violations (corruption 0.527); the complementarity of confidence-type and trust-type signals is architecture-independent. On CIFAR-10, where corruption genuinely reduces accuracy, the composed score is the strongest corruption detector outright (0.732, above its conformity factor alone at 0.723), and the model-side audit transfers: the label-noise-trained CIFAR model carries path trust 0.521 against 0.697 clean-trained, a 25% relative drop. One honest caveat: in the subset-direction OOD task the Fashion CNN’s composed score becomes anti-correlated (0.201), because sparse digit inputs activate few channels and short, well-trained paths read as trusted. An ablation normalizing each layer’s activity vector to mean one leaves the inversion unchanged (0.205; MNIST CNN likewise unchanged), ruling out the global activity level as the mechanism: the inversion sits in the participation pattern itself, and notably the softmax inverts on the same task as well (0.281), so it is a property of the model under subset-direction inputs rather than an artifact of trust propagation. Operationally, the composed score must not be used as an OOD filter in this direction; the diagnostics of Table 7.7 provide the gate.

Method	MNIST CNN		Fashion CNN		CIFAR-10
	Det.	OOD	Det.	OOD F→M	Det.
PaTAS (composed)	0.988	0.991	0.859	0.201	0.732
Conformity alone	1.000	1.000	0.994	0.481	0.723
Max softmax	0.662	0.967	0.711	0.281	0.700
MC Dropout	0.605	0.978	0.616	0.712	0.622
EDL	0.527	0.820	0.522	0.608	0.525

TABLE 7.10: Convolutional battery (mean over three evaluation seeds, dispersion at most ± 0.004): corruption detection on the mixed batch and OOD detection where a counterpart dataset is defined. Base accuracies 99.3%, 88.7%, 84.5%.

Sensitivity. Three ablations bound the framework’s sensitivity to its own choices. The trust-revision fusion operator is immaterial: averaging, cumulative, and weighted fusion agree on the converged trust mass and every downstream metric to within 0.002. The gradient-evidence threshold exhibits a broad plateau: trust mass and selective-prediction metrics are constant for $\epsilon \in [0.05, 1]$ on MNIST and degrade only at degenerate-small values, and the width ordering of Experiment 2 is invariant across $\epsilon \in \{0.02, 0.05, 0.1\}$; together with the scale-tied rule, this bounds the tuning burden to a single dimensionless constant. Finally, the per-sample scores are stable across evaluation seeds (dispersion at most ± 0.005 throughout).

Key findings – Experiment 4

F4.1 Attribution is explicit: input-space signals carry violation detection. The conformity opinions and the raw-pixel Mahalanobis baseline dominate every marginal-violation task, and removing the input opinions collapses the composed score to the output-only level. Trust propagation’s role there is composition, not detection.

F4.2 Propagation contributes where marginals are silent. Under marginal-preserving translation the composed score (0.781) beats its own input factor (0.678): atypical activation paths carry detection signal that no marginal input model can see.

F4.3 The model side is the unique contribution. The per-class poisoned trust gap, its channel separation, and the test-data-free audit (tripwire 31.7%, graded under informed opinions, robust to the dead-unit confound and to the scale-tied threshold) are signals no input-space or output-only method expresses.

F4.4 Confidence and trust are complementary by design. Softmax retains the better correct-versus-incorrect ranking on corrupted batches exactly because it ignores the violation that trust flags; the operational reading is gate on trust, then rank by confidence.

F4.5 Failure directions are principled and gated. Subset-direction OOD inverts the input factor (and, on the CNN, the composed score and softmax alike); the a-priori diagnostics predict both this failure and the GTSRB boundary, and the blended in-range trigger defeats all ten methods.

7.10 Discussion of PaTAS-TP Findings

$g \rightarrow$	$T_{n_i y_{\text{batch}}} \rightarrow$	$T_{y_{\text{batch}}} \rightarrow$	$T_{n_i Y_{\text{batch}}} \rightarrow$
0	Trusted	Trusted Uncertain DisTrusted	Trusted Uncertain Uncertain
$+\infty$	DisTrusted	Trusted Uncertain DisTrusted	DisTrusted (0,0.5,0.5) Uncertain

TABLE 7.11: Asymptotic behavior of $T_{n_i|Y_{\text{batch}}}$

PaTAS-TP is a framework for evaluating the runtime trustworthiness of NN outputs. Beyond dynamic inference assessment, it can also estimate a static trust opinion of the network itself. The core principle is that a trustworthy model should preserve trust: a fully trusted input producing a highly trusted output indicates that the model does not erode trust during inference. Accordingly, the overall model trustworthiness can be defined as the trust score of the PaTAS-TP feedforward function (PaTAS_{FF}) under a fully trusted input:

$$T(\text{NN}) = \text{PaTAS-TP}_{\text{FF}}((1, 0, 0)),$$

which quantifies how well trust is maintained throughout the network. This is the definition that we used to compare the IPTA for benign and adversarial inputs in Experiment 3, where the adversarial case used a 4×4 patch injection.

Although trust mass and accuracy are often correlated, our results reveal some divergences. A clear example is the MNIST experiment (Table 7.3): the 16-16 architecture achieves slightly better test accuracy than the single-layer 16-neuron network (95.84% vs. 95.13% under trusted data), yet its trust mass is substantially lower (0.7077 vs. 0.8620). The additional layer improves predictive performance while simultaneously eroding the trust signal, because each extra discounting step attenuates the evidence flowing through the network. This illustrates that accuracy and trust can evolve in opposite directions depending on architectural choices.

Interpretation of trust values, like accuracy, depends on the application. In high-stakes settings, a resulting trust score of (0.6, 0, 0.4) may be inadequate, whereas in less critical domains it may suffice. PaTAS-TP enables such contextual interpretation by providing a unified, interpretable metric that can be tracked over time, compared across models, or evaluated under varying conditions. Overall, PaTAS-TP can complement traditional metrics, for example, by signaling fragility even when accuracy appears high or by identifying stability where accuracy slightly decreases. This duality underscores its relevance in safety-critical contexts where accuracy alone can be misleading.

In practice, precise dataset trustworthiness estimates may be unavailable, especially for large training datasets where the provenance of individual samples is difficult to establish. PaTAS-TP can still operate in such cases by initializing dataset trust to a fully uncertain (vacuous) opinion. While this limits the use of prior trust evidence during training, it preserves the ability to propagate trust assessments at inference time. For a specific query, the circumstances under which the input features were collected are often easier to assess, making input-level trust more practical to obtain than dataset-level trust. PaTAS-TP can therefore support meaningful trust evaluation even when explicit dataset provenance is missing. Likewise, if trust in a particular input cannot be assessed, a fully uncertain opinion may be used as a fallback.

A key factor governing PaTAS-TP behavior is ϵ , the threshold used to classify gradients as positive or negative evidence. Its calibration ~~is critical~~ matters at the boundaries: if ϵ is too small relative to the prevailing gradient scale, most gradients are classified as large and treated as negative evidence, causing trust mass to collapse ~~and the framework to lose sensitivity to trust profile differences~~ (resembling the case $g \rightarrow +\infty$ in Table 7.11). ~~Conversely, if ϵ is too large,~~ nearly all gradients are treated as small ~~and the threshold loses discriminative power regardless of model behavior~~. Both failure modes are confirmed empirically in Table 7.1. ~~When profiling gradient distributions is impractical, a lightweight heuristic is to tie ϵ to the learning rate, providing a simple proxy for the expected gradient scale under the assumption that gradients decay roughly in proportion to the learning rate as training converges. A more principled approach sets ϵ relative to the empirical distribution of gradient magnitudes observed early in training. In practice, ϵ could be tuned per layer, since gradient scales naturally differ across layers, or scheduled alongside training to track the evolving gradient magnitude, or replaced by an adaptive scheme such as the denominator in and the plateau between them is broad (Section 7.9: metrics constant for $\epsilon \in [0.05, 1]$ on MNIST, orderings invariant across the sweep). The scale-tied rule of Section 7.6 operationalizes the principled resolution: setting $\epsilon_{\text{layer}} = c \cdot \text{median}(|\delta|)$ from each layer's own early gradient distribution instantiated thresholds spanning nearly two orders of magnitude~~

across the tasks studied here ($5 \cdot 10^{-5}$ on the MNIST input layer, $3 \cdot 10^{-3}$ on the cancer network) while preserving the audit tripwire and the trust orderings of the cancer grid with a single untuned constant $c = 0.5$; the magnitude of the resulting trust values shifts with c , so absolute trust masses are comparable only within one threshold convention, but every qualitative conclusion is threshold-independent. Fully adaptive schemes, such as per-parameter rescaling in the spirit of the Adaptive Moment Estimation (Adam) ~~, which tracks gradient magnitude per parameter and rescales automatically denominator, remain a natural refinement.~~

Finally, trust quantification for $T_{n_i|y_{\text{batch}}}$ in the Trust Update Algorithm (Alg. 6) relies on a simple gradient-counting procedure. Although computationally efficient, it fixes the uncertainty component of the resulting binomial opinion based on the number of input neurons to i . This fixed-uncertainty formulation may not be optimal in all cases, suggesting the need for adaptive quantification schemes. Similarly, $T_{n_i|\overline{y_{\text{batch}}}}$ is set to a fully vacuous opinion $(0, 0, 1)$, which, while consistent with the absence of evidence, may not fully leverage prior knowledge when available.

A subtlety visible in Fig. 7.7 is that the per-class neuron trust for patched digit 3 (0.74) exceeds that for clean digit 6 (0.43), even though patched digit 3 has much lower accuracy (19.80% vs. 97.70%). This is not inconsistent: the trust opinion associated with the class-3 output neuron reflects the reliability of the parameters on the digit-3 inference path, which are less poisoned than those on the digit-6 path. The path is relatively trustworthy; it is the input that is adversarially corrupted.

Experiment 4 turns these observations into an operational reading. Confidence and trust answer different questions: the softmax ranks correctness within the training distribution, and keeps doing so on corrupted batches precisely because it ignores the violation, while the trust factor flags the violation itself. Neither replaces the other, and the composed score of Section 7.8.2 makes the division of labor explicit: gate on the trust factor and the applicability diagnostics, then rank by confidence among trusted inputs. The same experiment also bounds what the audit measures: the dead-unit check shows the model-side trust drop is carried entirely by active units, so the audit reads training corruption, not network degeneration.

This distinction motivates the use of a softmax-weighted aggregation $\sum_i p_i t_i$, which weights each output neuron's trust mass by the model's predicted probability for that class. As shown in Fig. 7.8, this aggregation produces a more semantically aligned ranking: the trust mass for patched digit 3 (0.42) falls below that of clean digit 6 (0.46), correctly reflecting that the model's actual decision for a patched digit 3 image is unreliable. The softmax-weighted aggregator therefore combines path-level trust with decision-level confidence, making it better suited for scenarios where the goal is to assess trust in the final prediction rather than in the inference path alone.

7.11 Summary & Conclusions

This chapter presented PaTAS-TP, a framework for propagating trust through NNs in parallel with standard computation. The results are encouraging: trust assessments converge alongside model training, respect theoretical symmetry and invariance properties, and provide interpretable signals that complement accuracy in all ~~three-four~~ experimental settings. ~~We consider this an initial contribution whose viability is demonstrated on moderate-scale architectures; extending~~ The comparative evaluation makes the framework's value attributable rather than aggregate: input-space conformity carries violation detection and composes with path trust into a per-sample

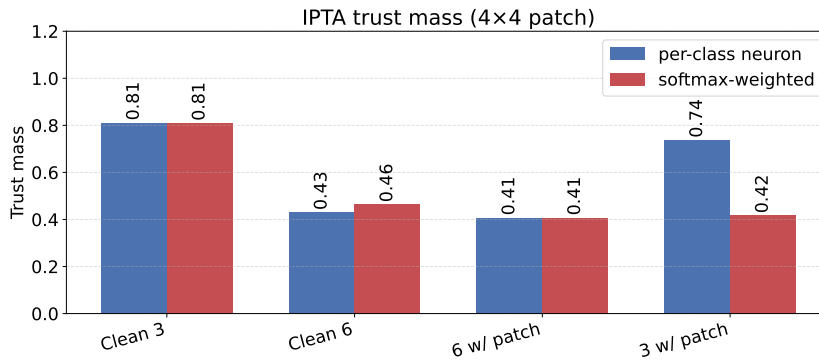


FIGURE 7.8: IPTA trust mass comparison between per-class neuron and softmax-weighted aggregation ($\sum_i p_i t_i$) for the NN trained on the poisoned dataset with patch size 4×4 . The softmax-weighted aggregation weights each output neuron’s trust mass by the model’s predicted probability, producing a ranking that better reflects the reliability of the final decision rather than the inference path alone.

score that fails safely, the propagation itself contributes detection signal where input marginals are silent, and the model-side trust yields per-class poisoning localization and a test-data-free training-quality audit that no input-space or output-only baseline expresses. The evaluation now covers residual convolutional networks at accuracies defensible for their domains (99.3% MNIST, 88.7% Fashion-MNIST, 84.5% CIFAR-10) through the activity-weighted conv IPTA; extending trust functions to transformer components and normalization layers, and scaling the evaluation to state-of-the-art models and top-tier training pipelines, such as transformer-based architectures or large-scale convolutional networks, is a natural and important direction remain natural directions for future work.

A known limitation of the current design is that it structurally penalizes deeper architectures. Each additional layer introduces an extra discounting step that attenuates the trust signal, so deeper networks inherit less trust from their inputs regardless of their predictive performance. This is a meaningful concern given that modern architectures are typically deep. Adapting the framework to compensate for depth, for instance by normalizing trust opinions with respect to path length or by using layer-wise trust scaling, is a direction for future work. The depth-normalized aggregation introduced in Section 7.3 compensates for the geometric component of this decay and makes trust values comparable across depths, and it is used alongside the raw aggregate in the audit of Experiment 4; a fully depth-invariant formulation of the propagation itself, rather than a corrected readout, remains future work. Two further limitations surfaced by the comparative evaluation deserve explicit mention. First, the trust functions combine opinions independently of weight and activation magnitudes: trust deliberately tracks the provenance and reliability of the quantities entering a computation rather than their functional influence, so an input the network effectively ignores can still contribute to the output opinion. Partial influence-aware steps exist, the softmax-weighted aggregation of Experiment 3 and the activity-weighted conv IPTA, but fully magnitude-aware trust functions remain open. Second, the conformity-based input opinions detect support violations, not support subsets; the subset-direction OOD failure is predicted by the a-priori diagnostics, and fusing a covariance- or neighborhood-based detector into the input opinion through the same quantification is the natural SL remedy.

A second limitation concerns the directional preservation of evidence established in Theorem 7.2 and its Remark. While theoretically sound, this property implies that PaTAS-TP cannot generate a non-zero distrust mass at the output when the input carries no distrust, that is, when $d_x = 0$, even if the model parameters are highly unreliable. In other words, parameter unreliability manifests as increased uncertainty rather than explicit distrust in the output opinion. This is a genuine restriction in safety-critical settings where a positive distrust signal would be more actionable than a high uncertainty mass. Relaxing this constraint, for instance by allowing parameter distrust to directly contribute a distrust component to the output, is another avenue for future work.

Finally, the gradient-counting procedure used in NODETRUST fixes the uncertainty component of the resulting binomial opinion based on the number of input neurons, which may not be optimal in all cases. Adaptive quantification schemes, as well as richer initialisation strategies for $T_{n_i|\overline{y}_{\text{batch}}}$ beyond the vacuous opinion, are further directions for improving the framework.

Bibliography

- [1] 5G Automotive Association (5GAA). *A Framework for Dynamic Trustworthiness Assessment in Cooperative and Automated Vehicles*. White Paper. Version 1.0. Approved by the 5GAA Board on 29 May 2025. 5G Automotive Association (5GAA), Sept. 1, 2025. URL: <https://5gaa.org/content/uploads/2025/08/5gaa-wi-trust4cav-white-paper-10072025-v3.pdf>.
- [2] *Architectural Specification of CONNECT Trust Assessment Framework, Operation and Interaction*. Tech. rep. Deliverable D3.1. EU Horizon Europe Project CONNECT. CONNECT Project, Horizon Europe, Mar. 2024. URL: <https://horizon-connect.eu/public-deliverables/>.
- [3] Koffi Ismael Ouattara et al. "An Optimized Framework for DSPG Synthesis and Trust Network Analysis with Subjective Logic". In: *Rules and Reasoning*. Ed. by Aidan Hogan et al. Cham: Springer Nature Switzerland, 2026, pp. 54–71. ISBN: 978-3-032-08887-1.
- [4] Koffi Ismael Ouattara et al. "On Subjective Logic Trust Discount for Referral Paths". In: *2024 27th International Conference on Information Fusion (FUSION)*. 2024, pp. 1–8. DOI: 10.23919/FUSION59988.2024.10706345.
- [5] Koffi Ismael Ouattara et al. "Assessing Trustworthiness of AI Training Dataset Using Subjective Logic - A Use Case on Bias". In: *Machine Learning and Principles and Practice of Knowledge Discovery in Databases*. Ed. by Irena Koprinska, João Mendes-Moreira, and Paula Branco. Cham: Springer Nature Switzerland, 2026, pp. 260–276. ISBN: 978-3-032-19096-3. URL: https://link.springer.com/chapter/10.1007/978-3-032-19096-3_17.
- [6] Koffi Ismael Ouattara et al. "Quantifying Dataset Trustworthiness from Labeling Bias Using Subjective Logic". In: (2025). URL: <https://ceur-ws.org/Vol-4132/short6.pdf>.
- [7] Koffi Ismael Ouattara et al. *PaTAS: A Framework for Trust Propagation in Neural Networks Using Subjective Logic*. 2025. arXiv: 2511.20586 [cs.AI]. URL: <https://arxiv.org/abs/2511.20586>.
- [8] Koffi Ismael Ouattara et al. "Quantifying Calibration Error in Neural Networks Through Evidence-Based Theory". In: *2025 28th International Conference on Information Fusion (FUSION)*. 2025, pp. 1–8. DOI: 10.23919/FUSION65864.2025.11124121.
- [9] Nikolaos Fotos et al. "Actions Speak Louder Than Words: Evidence-Based Trust Level Evaluation in Multi-agent Systems". In: *Information and Communications Security*. Ed. by Jinguang Han et al. Singapore: Springer Nature Singapore, 2026, pp. 255–273. ISBN: 978-981-95-3543-9.
- [10] Paul Monchot et al. "Input uncertainty propagation through trained neural networks". In: *Proceedings of the 40th International Conference on Machine Learning*. Ed. by Andreas Krause et al. Vol. 202. Proceedings of Machine Learning Research. PMLR, 23–29 Jul 2023, pp. 25140–25173. URL: <https://proceedings.mlr.press/v202/monchot23a.html>.

- [11] Ramon Astudillo and João Neto. "Propagation of Uncertainty Through Multilayer Perceptrons for Robust Automatic Speech Recognition." In: Aug. 2011, pp. 461–464. DOI: 10.21437/Interspeech.2011-196.
- [12] Mingxi Cheng, Shahin Nazarian, and Paul Bogdan. "There Is Hope After All: Quantifying Opinion and Trustworthiness in Neural Networks". In: *Frontiers in Artificial Intelligence* 3 (2020). ISSN: 2624-8212. DOI: 10.3389/frai.2020.00054. URL: <https://www.frontiersin.org/journals/artificial-intelligence/articles/10.3389/frai.2020.00054>.
- [13] W. H. Wolberg, O. L. Mangasarian, and W. N. Street. *Breast Cancer Wisconsin (Diagnostic) Data Set*. [https://archive.ics.uci.edu/ml/datasets/breast+cancer+wisconsin+\(diagnostic\)](https://archive.ics.uci.edu/ml/datasets/breast+cancer+wisconsin+(diagnostic)). UCI Machine Learning Repository. 1995.
- [14] Yann LeCun et al. "Gradient-based learning applied to document recognition". In: *Proceedings of the IEEE* 86.11 (1998), pp. 2278–2324.
- [15] Xinyun Chen et al. *Targeted Backdoor Attacks on Deep Learning Systems Using Data Poisoning*. 2017. arXiv: 1712.05526 [cs.CR]. URL: <https://arxiv.org/abs/1712.05526>.
- [16] Yarın Gal and Zoubin Ghahramani. *Dropout as a Bayesian Approximation: Representing Model Uncertainty in Deep Learning*. 2016. arXiv: 1506.02142 [stat.ML]. URL: <https://arxiv.org/abs/1506.02142>.
- [17] Murat Sensoy, Lance Kaplan, and Melih Kandemir. *Evidential Deep Learning to Quantify Classification Uncertainty*. 2018. arXiv: 1806.01768 [cs.LG]. URL: <https://arxiv.org/abs/1806.01768>.
- [18] Kimin Lee et al. "A Simple Unified Framework for Detecting Out-of-Distribution Samples and Adversarial Attacks". In: *Advances in Neural Information Processing Systems*. Vol. 31. 2018.
- [19] Yiyu Sun et al. "Out-of-Distribution Detection with Deep Nearest Neighbors". In: *International Conference on Machine Learning*. 2022.
- [20] Weitang Liu et al. "Energy-based Out-of-Distribution Detection". In: *Advances in Neural Information Processing Systems*. Vol. 33. 2020.
- [21] Han Xiao, Kashif Rasul, and Roland Vollgraf. *Fashion-MNIST: a Novel Image Dataset for Benchmarking Machine Learning Algorithms*. 2017. arXiv: 1708.07747 [cs.LG]. URL: <https://arxiv.org/abs/1708.07747>.
- [22] *German Traffic Sign Recognition Benchmark (GTSRB) Dataset*. Online. URL: <http://benchmark.ini.rub.de/>.